Republic of Tunisia

Ministry of Higher Education

and Scientific Research

Higher Private School of Engineering and Technology

TEK-UP

Final-Year Project Report

Submitted in fulfilment of the requirements for the

National Engineering Diploma in Applied Sciences and Technology

Speciality: Software Engineering

Prepared by

Nadim Jebali

Design and Development of a Multi-Vendor E-Commerce Marketplace

Professional Supervisor:	**Mr. Mohammed Aziz Chibani**	Software Engineer
Academic Supervisor:	**Ms. Ines Agrebi**	Lecturer

Academic Year 2025 - 2026

I authorise the student to submit the internship report for a defence.

Professional Supervisor, **Mr. Mohammed Aziz Chibani**

Signature and stamp

I authorise the student to submit the internship report for a defence.

Academic Supervisor, **Ms. Ines Agrebi**

Signature

Dedication

Acknowledgements

Contents

List of Figures

List of Tables

List of Abbreviations

- **AI** = Artificial Intelligence
- **API** = Application Programming Interface
- **CD** = Continuous Deployment
- **CI** = Continuous Integration
- **CI/CD** = Continuous Integration / Continuous Deployment
- **CRUD** = Create, Read, Update, Delete
- **DevOps** = Development and Operations
- **HTTP** = HyperText Transfer Protocol
- **HTTPS** = HyperText Transfer Protocol Secure
- **IaC** = Infrastructure as Code
- **JSON** = JavaScript Object Notation
- **JWT** = JSON Web Token
- **LLM** = Large Language Model
- **MVC** = Model-View-Controller
- **MoSCoW** = Must have, Should have, Could have, Won't have
- **ORM** = Object-Relational Mapping
- **PFE** = Projet de Fin d'Études
- **RBAC** = Role-Based Access Control
- **REST** = Representational State Transfer
- **SPA** = Single-Page Application
- **SQL** = Structured Query Language
- **UML** = Unified Modeling Language
- **URL** = Uniform Resource Locator
- **WYSIWYG** What You See Is What You Get

General Introduction

Online retail has become a central channel of commerce, yet in Tunisia the digital marketplace landscape remains fragmented. Sellers who wish to reach customers online are largely confined to two unsatisfying options: vertically integrated single-vendor stores, which they cannot join as independent merchants, and classified-ad sites, which list offers but provide no order management, no quality control and no buyer protection. The result is a market with weak tooling for content moderation, product discovery and trust, precisely the qualities that modern marketplaces are expected to guarantee.

It is in this context that the present project was carried out, as a final-year engineering project within the company **NeuralBey**. Its aim is to design and develop a **multi-vendor e-commerce marketplace**: a platform on which independent sellers open and operate their own stores within a shared environment governed by a central operator. The platform lets sellers onboard with minimal friction, publish products and fulfil orders, while administrators review seller applications, moderate content and curate featured selections. Its distinguishing capability is an **artificial-intelligence** verification pipeline that automatically screens stores and products to preserve buyer trust at scale, complemented by a conversational database assistant for operators and an AI-assisted storefront redesign for sellers. The system is engineered with a modern, fully containerised technical foundation and an infrastructure-as-code delivery pipeline.

To conduct the work in an organised and incremental manner, the project adopted the agile **Scrum** method. This report is structured into four chapters. The **first** presents the context and objectives of the project: the host organisation, the study of existing solutions, the proposed solution, the project-management methodology and the modelling language. The **second** analyses and specifies the requirements, actors, functional and non-functional requirements, use-case and class diagrams, the Scrum project backlog and sprint planning, the schedule, and the architecture and technical choices. The **third** and **fourth** chapters describe the realisation of the two successive releases, together with the tests and the deployment. A general conclusion finally draws the assessment of the work and its perspectives.

Chapter 1

Project Context and Objectives

Plan

Introduction

This first chapter lays the foundations of the project. We begin by presenting the host organisation within which the internship took place. We then set out the general context of the project through a study of the existing solutions, a critique of their limitations, and a description of the solution we propose. We continue with the project-management methodology adopted throughout the work, and we close with the modelling language used to specify the system.

1.1 Presentation of the Host Organisation

1.1.1 Company Overview

NeuralBey is a Tunisian technology company that delivers end-to-end engineering services across a broad spectrum of modern computing fields. The company positions itself as a partner that translates a client's vision into a working system, summarised by its own statement: *"From web and mobile apps to AI systems, IoT, embedded engineering, and 3D solutions, we build the technology that powers your vision."*

This breadth allows the company to take a project from initial concept through to deployment without external dependencies, and to combine disciplines that are usually siloed, for example, integrating artificial intelligence into conventional web platforms, or interfacing embedded devices with cloud services. Its portfolio reflects this versatility:

- **Web and mobile applications** for consumer products and enterprise tooling;
- **AI systems**, including model integration, automation pipelines and intelligent assistants;
- **IoT and embedded engineering**, where the company designs firmware and connected devices;
- **3D solutions** for product visualisation, AR/VR and digital twins.

Figure 1.1 shows the official logo of NeuralBey.

Figure 1.1: Logo of NeuralBey

1.1.2 Mission and Vision

NeuralBey operates as a multidisciplinary engineering studio. Its mission is to design and build reliable, modern software and connected systems, and to accompany each project from scoping through to delivery. Every project is staffed by engineers selected from the relevant domain (frontend, backend, AI, hardware, 3D), supported by a professional supervisor who follows the work throughout.

The company's vision rests on three principles:

- **Engineering excellence**, a working culture built on code review, iterative delivery and frequent stakeholder feedback, so that quality is continuous rather than verified only at the end.
- **Cross-disciplinary innovation**, combining fields that are usually separated, such as embedding artificial intelligence into ordinary web and mobile products.
- **End-to-end ownership**, delivering a project from concept to production, including the modern DevOps practices that keep a system maintainable beyond its first release.

These principles aligned naturally with the requirements of a final-year engineering project and shaped the way the present work was carried out.

1.1.3 Organigramme

Figure 1.2 illustrates the organisational structure of NeuralBey, built around a central engineering function organised by technical domain, a project-supervision and quality function, and a research-and-development unit.

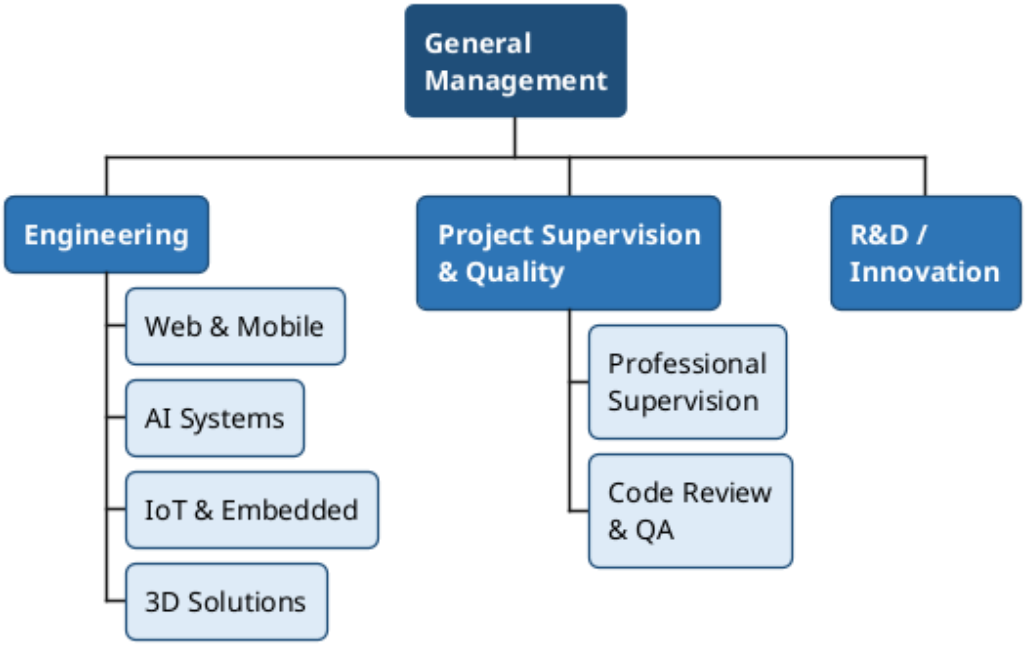

Figure 1.2: Organisational chart of NeuralBey

1.2 Project Presentation

The project originated from an internal need identified at NeuralBey: the Tunisian online-retail landscape lacks a modern, multi-vendor platform that is both seller-friendly and equipped with the automation features expected of contemporary marketplaces. Existing local solutions are typically single-vendor stores or classified-ad sites, and they offer limited tooling for moderation, content quality and discovery.

A *multi-vendor marketplace* is a platform on which independent sellers operate their own stores within a shared environment governed by a central operator. It differs from a **single-vendor store**, where the platform itself owns the inventory and the brand, and from a **classified-ad site**, where the platform merely lists offers and plays no part in transactions or quality control. A marketplace must therefore reconcile three concerns simultaneously: **seller autonomy** (each seller manages catalogue, branding and inventory), **buyer trust** (the platform guarantees a baseline of product authenticity and fulfilment) and **operator governance** (administrators onboard sellers, moderate content, suspend stores and curate featured selections).

Accordingly, three categories of target users were identified at the scoping stage:

- **Customer**, an end-user who browses the catalogue, places orders and tracks deliveries, expecting a fast, mobile-friendly experience with reliable search.
- **Seller**, a small or medium merchant who creates a store, manages a product catalogue and fulfils

orders, expecting low onboarding friction and protection against unfair competition from low-quality listings.

- **Administrator**, the platform operator who reviews seller applications, moderates flagged content, curates featured stores, and intervenes when a store violates the marketplace rules.

1.2.1 Study of Existing Solutions

To position the proposed solution, three categories of competing platforms were studied:

- **Pan-African marketplaces (Jumia).** Jumia Tunisia is the most visible regional marketplace. It supports multiple vendors at scale and offers a polished customer experience, but seller onboarding is heavyweight and the platform is not tailored to small Tunisian merchants.
- **Classified-ad sites (Tayara, Affariyet).** Tayara dominates the local classified-ad market. It offers excellent discovery for sellers, but performs no escrow, no order management, no integrated payment and no quality moderation.
- **Single-vendor stores (MyTek, Wiki, etc.).** These are vertically integrated retailers that operate their own catalogue and cannot be used by independent sellers.

1.2.2 Critique of Existing Solutions

From the study above, four gaps emerged:

- **No AI-assisted moderation.** On existing platforms, store and product verification are entirely manual, slow, inconsistent, and impractical at scale.
- **Heavyweight seller onboarding.** Pan-regional players require contractual paperwork unsuitable for small merchants.
- **Limited operator tooling.** Classified-ad sites have weak admin dashboards; single-vendor stores have none for external sellers.
- **Outdated technical foundations.** Several platforms still rely on monolithic stacks without modern CI/CD or infrastructure-as-code, making evolution costly.

Table 1.1 summarises the comparison along the five criteria that drove the design of the proposed solution. A check mark (✓) indicates that the criterion is fully supported, a cross (✗) that it is absent, and a tilde (∼) partial support.

Table 1.1: Comparative analysis of competing platforms

Criterion	Jumia	Tayara	Proposed Solution
Multi-vendor model	✓	∼	✓
AI verification	✗	✗	✓
Admin dashboard	∼	✗	✓
CI/CD pipeline (open)	✗	✗	✓
Lightweight onboarding	✗	✓	✓

1.2.3 Proposed Solution

The proposed platform is a multi-vendor marketplace designed around three pillars: a self-service seller experience, AI-assisted content verification, and a modern, fully containerised deployment. It is built as a single-page **React** application backed by a modular **NestJS** API, with **MariaDB** as the relational store and **n8n** acting as an orchestration layer for AI tasks. Authentication is handled by **Better-Auth** with email verification, and **Prisma** is used as the ORM. The full system is packaged with Docker Compose, provisioned on DigitalOcean via Terraform, and delivered through a GitHub Actions CI/CD pipeline.

Seven differentiators set the platform apart from the solutions surveyed above:

- **AI verification of stores and products** via an n8n workflow that calls a hosted large-language model and writes back a review flag together with a textual rationale.
- **Lightweight seller onboarding** through a seller application form reviewed by administrators, with a free tier offering a single store at zero cost.
- **Integrated administration tooling** (review, suspend, curate, cascade-delete) implemented as first-class features rather than database admin scripts.
- **Conversational data access** via a database-aware chatbot built on n8n, allowing administrators to query the platform in natural language.
- **AI-assisted storefront redesign**, sellers can regenerate their entire store visual identity from a natural-language prompt, powered by an n8n workflow and a hosted large-language model.
- **Store-wide sale promotions** that propagate a percentage discount automatically across product cards, store listings and search results, applying the discounted price at checkout without per-product editing.

- **Modern DevOps foundations**, a pnpm monorepo, Docker Compose, Terraform on DigitalOcean and GitHub Actions CI/CD, demonstrating production-grade engineering practices end to end.

The scope was agreed with the professional supervisor and bounded to keep the deliverables clear:

In scope: account creation and email verification; the seller application workflow with admin review; store and product CRUD with image upload; tag-based product recommendations with a top-seller fallback; order placement, items and history; secure online card payment at checkout; admin moderation panels (store suspension, cascade-delete); AI verification and chat assistant via n8n; store-wide percentage sales with automatic badge propagation; AI-assisted storefront redesign; a premium tier unlocking additional stores; and infrastructure-as-code deployment.

Out of scope (deferred to future iterations): a native mobile application; advanced fraud detection; on-premise LLM hosting; multi-currency support; and a full-text search engine.

1.3 Project-Management Methodology

Every software project requires a methodological framework to organise work, manage priorities and guarantee the delivery of a quality product. This section justifies the choice of an agile method, **Scrum**, and presents its process, roles, events and artefacts.

1.3.1 Classical vs Agile Approaches

Two major families of methods have historically competed: the **classical** (predictive) approach and the **agile** (adaptive) approach. The classical *Waterfall* model divides the project into strictly sequential phases, requirements, design, development, testing, delivery, each fully validated before the next begins. It suits projects whose requirements are stable and known in advance, but it shows its limits as soon as the context evolves: requirements are frozen at launch, the client sees a working product only at the very end, and defects are detected too late to address without significant impact.

Agility, born from the *Agile Manifesto* (2001) [1], champions collaboration, adaptability and the continuous delivery of value. Rather than planning the entire project upfront, work is organised into **short iterations**, each producing a functional and tested increment. Requirements are refined continuously from feedback, value is delivered from the first iterations, and risks are identified and addressed early.

1.3.2 Main Types of Agile Methodologies

Several frameworks implement the agile principles, each with a different emphasis, as summarised in Table 1.2.

Table 1.2: Main types of agile methodologies

Framework	Emphasis
Scrum	Organises work into time-boxed *sprints* around three accountabilities and a small set of events; the most widely adopted agile framework.
Kanban	Visualises the flow of work on a board and limits work-in-progress, favouring continuous flow over fixed iterations.
Extreme Programming (XP)	Focuses on engineering practices such as pair programming, test-driven development and continuous integration.

1.3.3 Why Scrum?

Scrum [2] was selected because it offers the clearest framework for delivering a substantial product in successive, demonstrable increments while keeping requirements open to change. Its sprint rhythm gives the project a regular cadence of planning and review, its backlog provides a single prioritised source of truth for the work, and its review events give the professional supervisor, acting as the stakeholder, frequent opportunities to steer the product. These properties fit a project whose scope was expected to evolve through the internship.

1.3.4 The Scrum Process

Scrum builds the product through a series of **sprints**: short, fixed-length iterations at the end of which a potentially shippable increment is delivered. Each sprint draws its work from the *Product Backlog* into a *Sprint Backlog*, the work is built and tested during the sprint, and the resulting increment is reviewed with the stakeholder before the next sprint begins. Figure 1.3 illustrates this cycle.

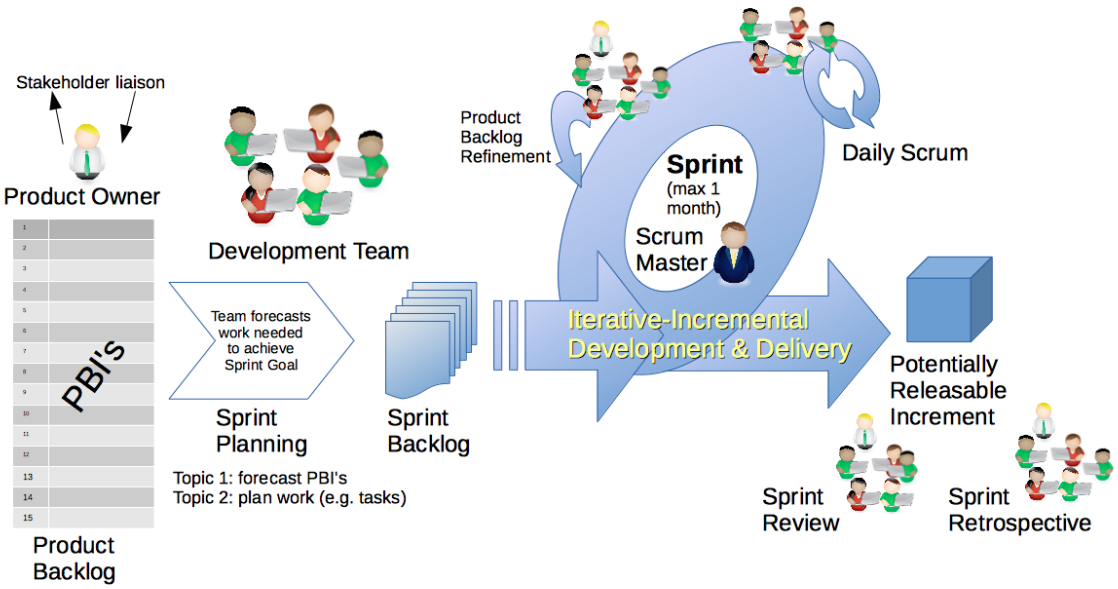

Figure 1.3: The Scrum framework: from the Product Backlog to a shippable increment

1.3.4.1 Scrum Roles

Scrum defines three accountabilities, mapped to the present project as follows:

- **Product Owner**, owns the product vision and prioritises the backlog. This role was held by the professional supervisor, **Mr. Mohammed Aziz Chibani** (NeuralBey), who also provided architectural guidance.
- **Scrum Master**, ensures the framework is applied and removes impediments. In a single-developer context, this role was assumed by the student.
- **Development Team**, builds the increment. The entire technical realisation, analysis, design, implementation, testing and deployment, was carried out by the student, **Nadim Jebali**.

1.3.4.2 Scrum Events

The work was punctuated by Scrum's events:

- **Sprint Planning**, selecting, at the start of each sprint, the backlog items to be delivered.
- **Daily Scrum**, a short daily checkpoint to track progress and surface impediments.
- **Sprint Review**, demonstrating the increment to the Product Owner and gathering feedback.
- **Sprint Retrospective**, reflecting on the process to improve the next sprint.

1.3.4.3 Scrum Artefacts

Three artefacts ensured transparency of the work:

- **Product Backlog**, a prioritised list of all expected functionalities, maintained as user stories with MoSCoW priorities and complexity estimates (detailed in Chapter 1.4).
- **Sprint Backlog**, the subset of backlog items committed to the current sprint.
- **Increment**, a working, demonstrable version of the product delivered at the end of each sprint.

1.4 Modelling Language

To analyse and design the system, the project relies on the **Unified Modeling Language** (UML) [3], the standard graphical notation for specifying and visualising the artefacts of a software system. UML offers complementary diagrams that capture both the structural and the behavioural views of a system: *use-case diagrams* express the functional requirements from the actors' perspective, *class diagrams* describe the static domain model, and *sequence* and *activity diagrams* depict dynamic behaviour. The next chapter uses these diagrams to formalise the requirements and the design of the platform.

Figure 1.4: The Unified Modeling Language (UML) logo

Conclusion

This chapter established the foundations of the project. It introduced NeuralBey, the host organisation, and the business motivation behind building a modern Tunisian multi-vendor marketplace. A study of existing platforms revealed four gaps, absent AI moderation, heavyweight onboarding, weak admin tooling and outdated technical foundations, that the proposed solution directly addresses through its key differentiators and bounded scope. Finally, the Scrum methodology and the UML modelling language that frame the rest of the work were presented. The following chapter formalises the requirements derived from this context.

Chapter 2

Requirements Analysis and Specification

Plan

Introduction

This chapter identifies the three system actors, formalises the functional and non-functional requirements of the platform, and illustrates them with UML use-case and class diagrams. It then presents the Scrum management of the project, the team, the product backlog and the sprint planning, followed by the schedule and the architecture and technical choices that underpin the realisation.

2.1 Requirements Specification

2.1.1 Identification of Actors

The platform recognises three actor categories. They are distinct *roles* stored on the `User` table rather than distinct user records: a single account can be promoted from *customer* to *seller* once a seller application is approved, and an administrator account is created automatically on first boot.

- **Customer.** The default role assigned upon registration. A Customer can browse the catalogue, search products, view individual stores, manage a cart, place an order across multiple sellers in a single checkout, and review their order history. Any Customer can submit a seller application and, once approved, gain the Seller role on the same account. Anonymous visitors share the read-only abilities of the Customer but cannot maintain a cart or place an order.
- **Seller.** A Customer whose seller application has been approved by an administrator. A Seller can create and edit one store under the free tier (or several under the premium tier), populate it with products, upload product images, attach product tags, choose a visual template, and consult the orders specific to their store. Sellers cannot review or moderate the work of other sellers.
- **Administrator.** Created automatically by the backend at first boot and not exposed through registration. The Administrator reviews seller applications, browses every store and product, suspends or deletes stores (with cascading deletion), overrides automated AI verification decisions, curates featured content, manages categories, and consults the platform through a database-aware AI chat assistant.

2.1.2 Functional Requirements

The functional requirements are grouped into eight domains. Each domain corresponds to a backend NestJS module and to a matching set of pages on the React frontend.

User Registration and Authentication.

- The system shall allow a visitor to create an account with an email address and password.
- The system shall send a verification email containing a one-time link upon successful registration.
- The system shall prevent unverified accounts from creating a store or placing an order.
- The system shall authenticate returning users via Better-Auth sessions and issue an HTTP-only cookie.
- The system shall allow the user to log out and invalidate the corresponding session.
- The system shall enforce role-based access control on every non-public route through guards and a `@Roles()` decorator.

Store Management.

- A Seller shall create a store with a name, description, logo and template, and edit it at any time.
- A free-tier Seller shall be limited to a single store; a premium-tier Seller shall operate multiple stores.
- Each store shall expose a public storefront URL identified by a human-readable slug derived from its name, with uniqueness enforced and the slug regenerated when the name changes.
- The Seller shall customise the storefront across seven independent sections (theme, announcement bar, banner, header, product grid, sidebar, footer) through a WYSIWYG visual editor, with undo/redo and selectable presets.
- Customisation features shall be tiered: free accounts use a curated subset, while premium accounts unlock the full set; on premium expiry, existing customisation is preserved but no longer editable.
- A Seller shall activate a store-wide percentage sale that propagates a discounted price and a sale badge across product cards, the store card and search results, and applies at checkout.
- A Seller shall trigger an AI-assisted storefront redesign by submitting a natural-language prompt, which generates a complete customisation applied live to the editor for review.

Product Management.

- A Seller shall create, edit and delete products in their own store(s).
- Each product shall accept one or more images of validated type and size, and an arbitrary number of tags used by the recommendation engine.
- Each product shall carry an `aiReviewed` flag set by the verification pipeline and an `adminReviewed` flag set on manual override.
- Products shall be browsable by category and sub-category.

Order System.

- A Customer shall add and remove items in a client-side cart.
- Checkout shall create a single order even when the cart spans multiple stores, with one order item per product line, and trigger a confirmation email.
- Each Customer shall consult their order history and each Seller the orders that include at least one of their products.

Administration.

- The Administrator shall access a dashboard listing pending seller applications, recent activity and platform statistics.
- The Administrator shall approve or reject seller applications, granting the Seller role on approval.
- The Administrator shall suspend, reactivate or delete any store, with cascading deletion of its products and orders, and override any AI verdict.

AI Verification.

- Once its images are uploaded, every store and product shall be submitted to an n8n verification workflow that screens each image together with the listing text using a hosted multimodal large-language model.
- The workflow shall return a decision (*approve* or *flag*) with a textual rationale, written back to the entity through a callback endpoint.
- Flagged entities shall appear in the moderation queue for administrative review.

Product Suggestions.

- On a product page, the system shall present cross-store suggestions, a cheaper "same item" offer when one exists and a set of substitute alternatives, distinct from the tag-based recommendations of the landing page.
- Suggestions shall be generated live for each product view by an n8n workflow (a hosted large-language model with a read-only database tool returning structured product identifiers), so that a newly listed product is eligible immediately, with no precomputed data to go stale.
- The backend shall validate every suggested identifier at serve time, approved, not banned, available, in stock and never the viewed product, with a "better deal" kept only when genuinely cheaper, and

shall fall back to a deterministic catalogue query (same category, ranked by product-name similarity and tag overlap) on any workflow error, timeout or empty result, so the section is never empty.

Featured Content Curation.

- The Administrator shall mark any store or product as *featured*, promoting it on the landing page.
- In the absence of featured products for a tag, the recommendation engine shall fall back to top-selling products of that tag.

2.1.3 Non-Functional Requirements

Beyond pure functionality, the deliverable must exhibit the qualities summarised in Table 2.1, prioritised with the professional supervisor during the requirements review.

Table 2.1: Non-functional requirements

Criterion	Description
Performance	Catalogue and store pages render in under one second on a 10 Mbit/s connection; typical API endpoints respond in under 300 ms.
Security	Passwords hashed by Better-Auth; sessions in HTTP-only, `SameSite=Lax` cookies; HTTP security headers via Helmet; uploads validated by type and size; every protected route gated by an auth guard and the `@Roles()` decorator, with object-level ownership enforced in the service layer. Required secrets are validated at start-up.
Scalability	The backend is stateless and horizontally scalable behind a reverse proxy; uploads live on a volume that can migrate to object storage with no code change.
Availability	The production stack runs under Docker Compose with restart policies; deployment is zero-downtime through rolling container replacement.
Usability	The frontend follows accessible component patterns, is fully responsive, and uses slug-based, human-readable URLs.
Maintainability	The codebase is split into bounded NestJS modules, the schema is versioned through Prisma migrations, and the infrastructure is reproducible through Terraform.
Observability	Application logs are emitted to `stdout` and aggregated by Docker; the chat assistant offers a queryable view of the database for diagnostics.

2.2 Global Use-Case Diagram

Figure 2.1 groups the principal use cases of the platform around its three actors. Two dotted dependencies illustrate the cross-actor flows on which the rest of the analysis relies: a seller application *triggers* an administrative review, and product creation is automatically *verified by AI*.

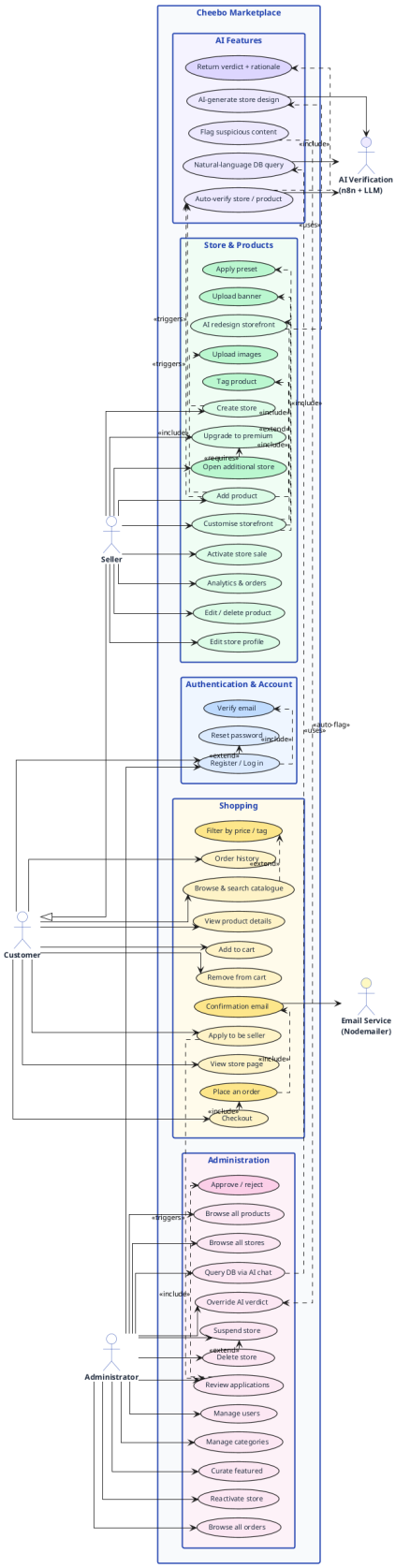

Figure 2.1: Global use-case diagram

2.3 Class Diagram

Figure 2.2 presents the static domain model. A single `User` entity carries the customer, seller and administrator roles, together with the premium-subscription and Stripe billing attributes (`isPremium`, `premiumExpiresAt` and the Stripe customer and subscription identifiers); a Seller owns one or more `Store` entities, each holding `Product` entities that are organised by `Category` and `Tag`; orders are modelled by an `Order` aggregate and its `OrderItem` lines, allowing a single checkout to span several stores.

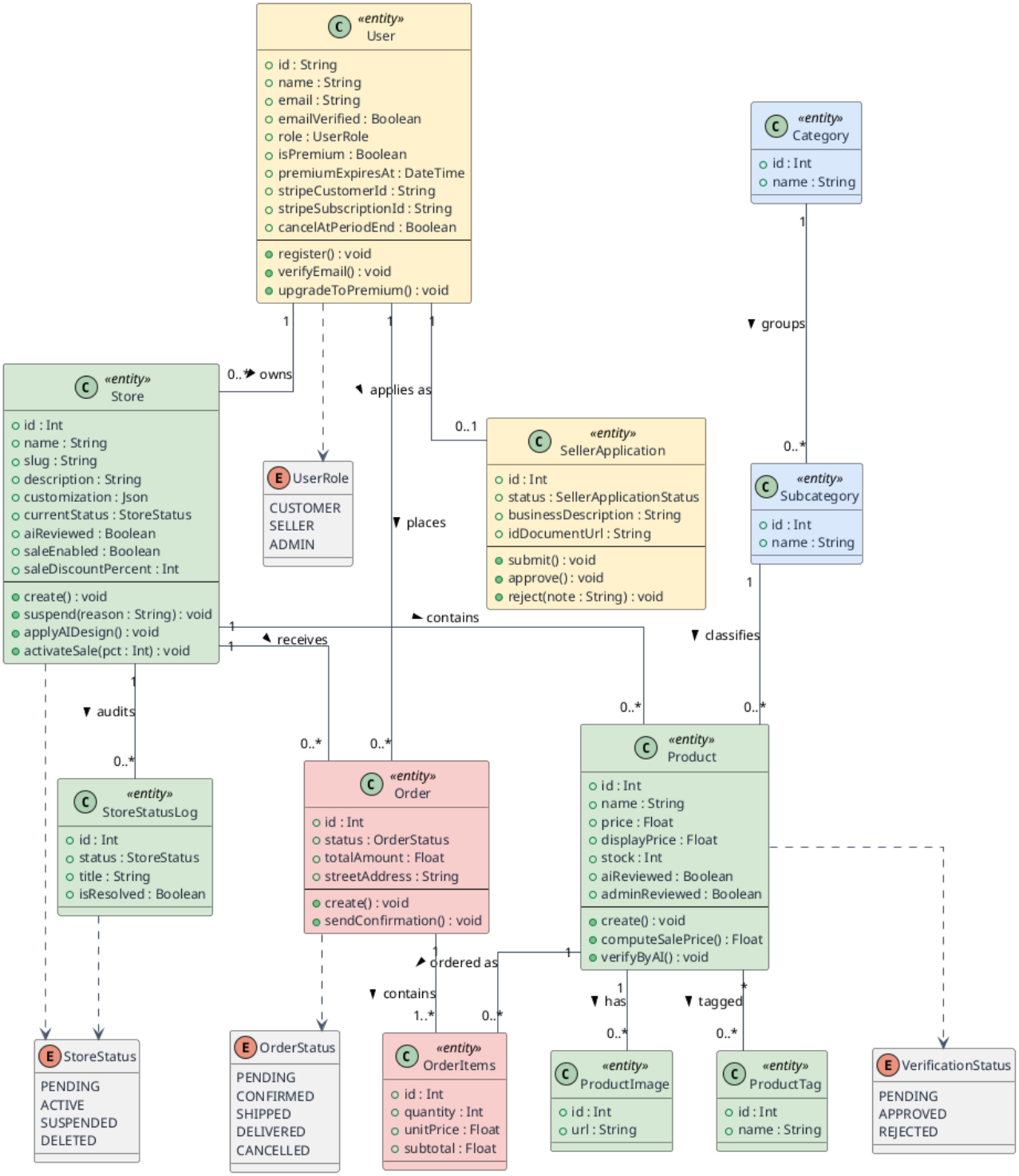

Figure 2.2: Class diagram of the platform domain model

2.4 Project Management with Scrum

2.4.1 Scrum Team

The Scrum accountabilities were distributed as follows:

- **Product Owner:** Mr. Mohammed Aziz Chibani (NeuralBey), who owned the product vision and prioritised the backlog.

- **Scrum Master and Development Team:** Nadim Jebali (the student), responsible for applying the framework and for the entire technical realisation.

2.4.2 MoSCoW Priority Legend

Each user story carries a **MoSCoW** priority and a **complexity** estimate, defined in Table 2.2.

Table 2.2: MoSCoW priority and complexity legend

MoSCoW Priority		
Must	*Must have*	Essential feature, required for the MVP.
Should	*Should have*	Important but not blocking.
Could	*Could have*	Desirable, included if time permits.
Won't	*Won't have*	Out of scope for this version.
Complexity		
1	Simple	Straightforward implementation, minimal effort.
2	Moderate	Average complexity, moderate effort.
3	Complex	Significant effort, elaborate logic.
5	Very complex	High effort, advanced technical challenge.

2.4.3 Product Backlog

Table 2.3 presents the product backlog as user stories following the template *"As a <role>, I want <action> so that <benefit>"*. The columns **P** and **Cx** indicate the MoSCoW priority and the estimated complexity.

Table 2.3: Product backlog, user stories with MoSCoW priority and complexity

Feature	ID	User Story	P	Cx
F1 Authentication	US01	As a visitor, I want to create an account with my email so that I can access the platform.	**Must**	**2**
	US02	As a visitor, I want a verification email so that I can activate my account securely.	**Must**	**2**
	US03	As a user, I want to sign in with my credentials so that I can access my dashboard.	**Must**	**1**

Continued on next page

Feature	ID	User Story	P	Cx
	US04	As a user, I want to sign out so that I can protect my account on a shared device.	**Must**	**1**
	US37	As a user, I want to reset my password by email when I forget it so that I can regain access to my account.	**Should**	**2**
F2 Store Management	US05	As a seller, I want to create a store with a name, description and logo so that customers can recognise my brand.	**Must**	**2**
	US06	As a seller, I want my store URL to use a readable slug so that it is easy to share.	**Must**	**2**
	US07	As a seller, I want to customise my storefront across seven sections through a WYSIWYG editor so that I can style my store live.	**Should**	**5**
	US08	As a seller, I want to apply a preset so that I can get a professional look quickly.	**Should**	**2**
	US09	As a premium seller, I want to operate multiple stores so that I can segment my offerings.	**Should**	**2**
	US34	As a seller, I want a store-wide percentage sale so that all my products show discounted prices and a badge automatically.	**Should**	**2**
F3 Product Management	US10	As a seller, I want to create, edit and delete products so that I can keep my catalogue up to date.	**Must**	**2**
	US11	As a seller, I want to upload multiple product images so that customers can see my products clearly.	**Must**	**2**
	US12	As a seller, I want to assign tags to products so that the recommendation engine can surface them.	**Should**	**1**
	US13	As a customer, I want to browse products by category so that I can discover relevant items.	**Must**	**1**

Continued on next page

Feature	ID	User Story	P	Cx
	US41	As a customer, I want to see on a product page a cheaper offer for the same item and relevant alternatives across stores so that I can compare and avoid overpaying.	**Could**	**3**
F4 Order System	US14	As a customer, I want to add products to a cart so that I can purchase multiple items at once.	**Must**	**2**
	US15	As a customer, I want to check out across multiple stores in a single order so that I avoid separate transactions.	**Must**	**3**
	US16	As a customer, I want a confirmation email after ordering so that I have a record of my purchase.	**Must**	**1**
	US17	As a seller, I want to view orders containing my products so that I can fulfil them.	**Must**	**2**
	US36	As a customer, I want to pay securely for my order online so that my purchase is confirmed instantly.	**Must**	**3**
F5 Administration	US18	As an admin, I want to review seller applications so that I can approve legitimate merchants.	**Must**	**2**
	US19	As an admin, I want to suspend or delete any store so that I can enforce marketplace rules.	**Must**	**2**
	US20	As an admin, I want to curate featured stores and products so that the landing page promotes quality.	**Should**	**1**
	US21	As an admin, I want to manage categories so that the catalogue stays organised.	**Must**	**1**
F6 AI Verification	US22	As the system, I want to verify every new store and product via an LLM so that low-quality content is flagged automatically.	**Should**	**3**

Continued on next page

Feature	ID	User Story	P	Cx
	US23	As an admin, I want to override any AI verdict so that I remain the final authority.	**Must**	**1**
	US24	As a customer, I want an AI-verified badge on trusted products so that I can shop with confidence.	**Should**	**1**
	US35	As a seller, I want to regenerate my storefront from a natural-language prompt so that I get a professional design without manual work.	**Could**	**3**
F7 Premium Tier	US25	As a seller, I want to subscribe to a premium plan so that I can unlock advanced features.	**Should**	**3**
	US26	As a premium seller, I want all customisation controls so that I can fully personalise my store.	**Should**	**3**
	US27	As the system, I want to preserve premium customisation after expiry so that the store keeps its design.	**Could**	**2**
F8 CI/CD & Deployment	US28	As a developer, I want every service in a Docker container so that environments are identical.	**Must**	**2**
	US29	As a developer, I want images built and pushed automatically on every push to main so that deployments are current.	**Must**	**2**
	US30	As a developer, I want the droplet provisioned via Terraform so that infrastructure is reproducible.	**Should**	**3**
	US31	As a developer, I want deployment over SSH on every push to main so that updates reach production automatically.	**Must**	**3**
	US32	As a developer, I want secrets managed through CI secrets so that no credentials live in the repository.	**Must**	**1**

Continued on next page

Feature	ID	User Story	P	Cx
	US33	As a developer, I want DNS records updated automatically to the new droplet IP so that the application is always reachable.	**Should**	**2**
F9 Account Management	US38	As a user, I want to view and edit my profile and change my password so that I can keep my account details current and secure.	**Must**	**2**
	US39	As a customer, I want to consult my purchase history so that I can review my past orders.	**Should**	**1**
F10 Appearance & Theming	US40	As a user, I want to switch between a light and a dimmed dark theme, or follow my system preference, so that I can browse comfortably in low light.	**Could**	**2**

2.4.4 Analysis Summary

The product backlog comprises **41 user stories** across **10 features**. The bulk are *Must* or *Should* priorities forming the marketplace core, while a few advanced capabilities (AI redesign, premium-customisation preservation) are *Could* items scheduled only once the essentials are delivered. Complexity estimates concentrate effort on the storefront editor, the multi-store checkout, the AI verification pipeline and the deployment automation.

2.4.5 Sprint Planning per Release

The backlog was organised into **two releases of two sprints each**. The first release delivers the social and transactional foundations; the second adds intelligence, monetisation and the deployment pipeline. Table 2.4 maps each sprint to its goal, user stories and estimated duration. Each sprint runs about one to one-and-a-half months, so the two releases together span roughly five to six months.

Table 2.4: Sprint planning across the two releases

Release	Sprint	Goal	User stories	Est. duration
Release 1	Sprint 1	Authentication and seller onboarding	US01 account creation, US02 email verification, US03 sign-in, US04 sign-out, US18 seller-application review, US37 password reset	~1 month
	Sprint 2	Store and product management	US05 store creation, US06 readable store slug, US07 storefront WYSIWYG editor, US08 style presets, US09 multiple stores (premium), US10 product CRUD, US11 product images, US12 product tags, US13 category browsing, US34 store-wide sales	~1.5 months
Release 2	Sprint 3	Orders, payment, account and administration	US14 shopping cart, US15 multi-store checkout, US16 order-confirmation email, US17 seller order view, US19 store suspension/deletion, US20 featured-content curation, US21 category management, US36 secure card payment, US38 profile management, US39 purchase history	~1.5 months
	Sprint 4	AI verification, product suggestions, premium tier, theming and DevOps	US22 AI store/product verification, US23 AI verdict override, US24 verified badge, US25 premium subscription, US26 premium customisation controls, US27 premium preservation on expiry, US28 service containerisation, US29 CI image builds, US30 Terraform provisioning, US31 SSH deployment, US32 CI-managed secrets, US33 DNS automation, US35 AI storefront redesign, US40 light/dark theming, US41 cross-store product suggestions	~1.5 months
			Total estimated duration	**~5–6 months**

2.5 Gantt Diagram

Figure 2.3 presents the project schedule over the internship: an inception phase, the four sprints grouped into two releases with their delivery milestones, a tests-and-deployment phase, and a report-writing activity running in parallel throughout.

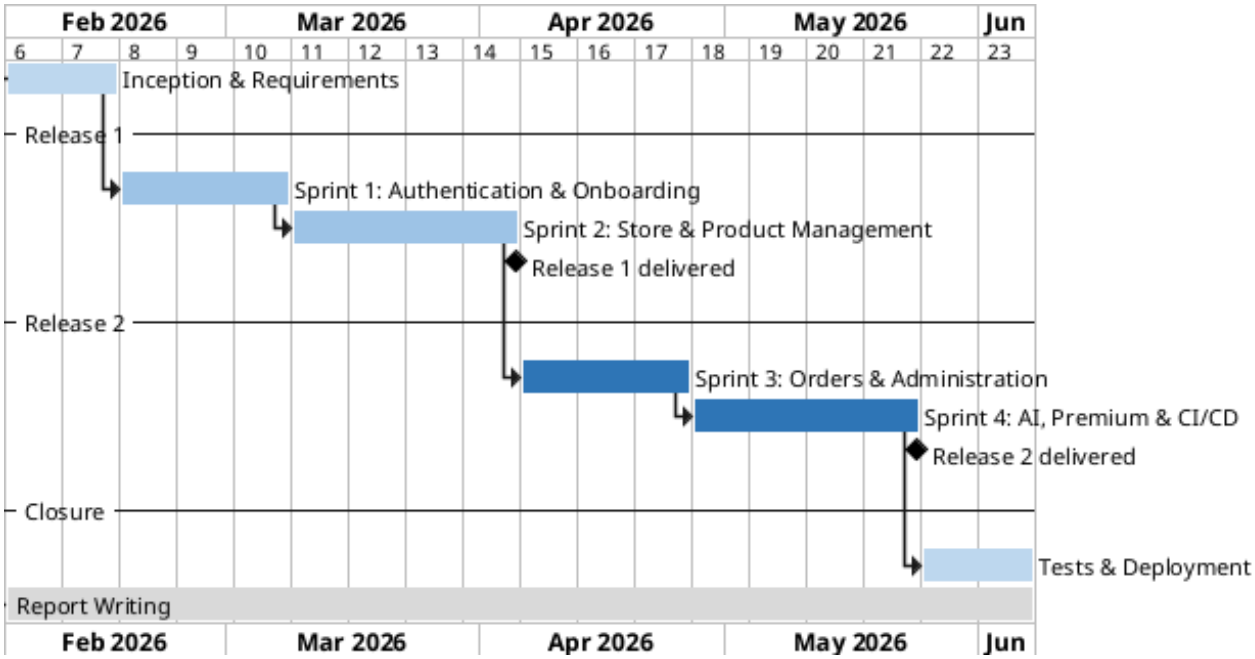

Figure 2.3: Project Gantt chart (two releases, four sprints)

2.6 Architecture and Technical Choices

2.6.1 Logical Architecture

The platform follows a classic **three-tier architecture** that separates the user interface (presentation), the business logic (application) and the persistent data (data) into independent layers, with an **n8n** automation layer orbiting the backend to host the AI workflows. End users reach the platform over HTTPS; a reverse proxy serves the React bundle and forwards `/api/*` requests to the NestJS API, which persists state to MariaDB through Prisma and exchanges webhooks with n8n. Figure 2.4 shows the topology.

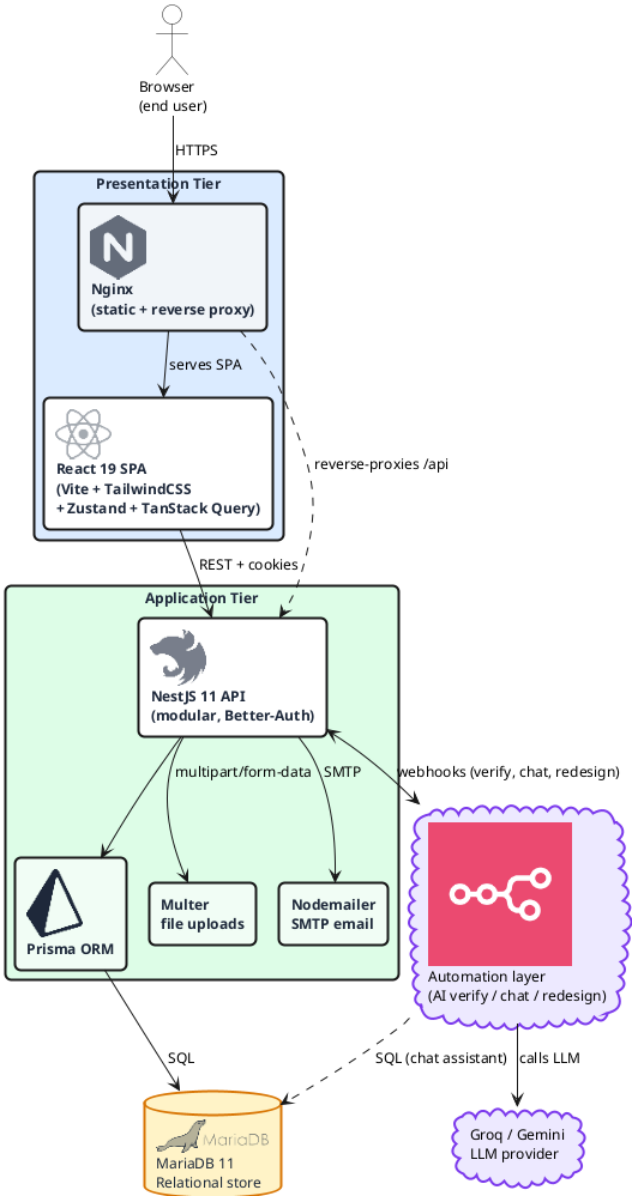

Figure 2.4: Logical architecture (three tiers and the n8n automation layer)

2.6.2 Physical Architecture

In production, every component runs as a **Docker** container orchestrated by Docker Compose on a single DigitalOcean droplet. A **Caddy** reverse proxy terminates TLS with automatically renewed Let's Encrypt certificates and routes traffic to the frontend, backend and n8n containers over an internal bridge network; uploads and the n8n state persist on Docker volumes. Figure 2.5 details the container topology.

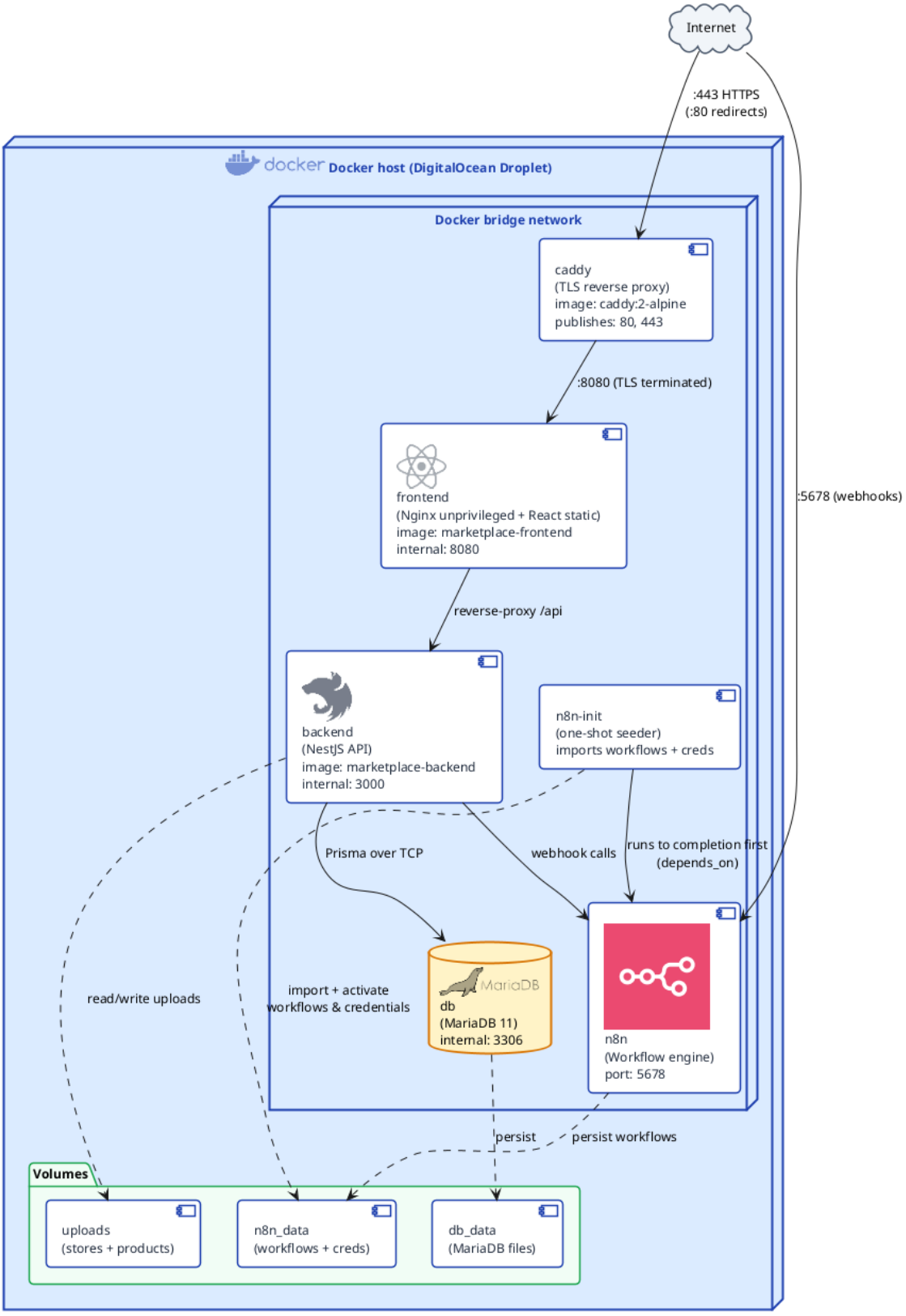

Figure 2.5: Physical architecture (Docker Compose services)

2.6.3 Artificial-Intelligence Models

The intelligent features are powered by hosted large-language models, orchestrated through n8n so that providers can be swapped without touching the backend. Table 2.5 lists the model used by each

service.

Table 2.5: AI models used per service

Service	Model / provider
Store and product verification	Multimodal LLM (Google Gemini [4]) screening images and listing text
AI storefront redesign	Groq [5] `llama-3.3-70b-versatile` generating the storefront customisation
Database chat assistant	Hosted LLM translating natural-language questions into safe database queries
Product suggestions	Hosted LLM (Google Gemini [4] / Groq [5]) reasoning over the catalogue through a read-only SQL tool, returning structured product ids

2.6.4 Tools, Frameworks and Languages

The system is implemented in **TypeScript** end to end. Table 2.6 lists the tools used throughout the project, covering source control, project tracking, development, containerisation, infrastructure provisioning, workflow automation and API testing, while Table 2.7 summarises the main frameworks and libraries.

Table 2.6: Tools used throughout the project

Logo	Tool	Role
	GitHub [6]	Version control, pull-request review and CI/CD via GitHub Actions.
	Notion [7]	Backlog management, task board and record of user stories.
	VS Code [8]	Primary IDE, extended for TypeScript, Prisma, ESLint and Git.
	Docker [9]	Containerisation of all services for dev/prod parity.
	Terraform [10]	Infrastructure-as-code provisioning of the DigitalOcean resources.
	n8n [11]	Visual workflow automation orchestrating the AI features.
	Postman [12]	Manual testing and documentation of the REST endpoints.
	pnpm [13]	Fast, disk-efficient package manager for the monorepo.

Table 2.7: Frameworks and languages

Technology	Role
TypeScript [14]	Statically-typed language used across the frontend and the backend.
React [15]	Single-page application library for the customer, seller and admin interfaces.
NestJS [16]	Modular Node.js framework structuring the backend into feature modules.
Prisma [17]	ORM and migration tool over MariaDB.
Better-Auth [18]	Authentication library handling sessions and email verification.
Docker	Container runtime packaging every service.
Terraform	Declarative provisioning of the cloud infrastructure.
n8n	Workflow engine hosting the AI orchestration.

Conclusion

This chapter formalised the requirements of the platform. The three actors were identified, the functional requirements grouped into eight domains and the non-functional requirements stated, then illustrated with the global use-case diagram and the class diagram. The Scrum management of the project was presented, the team, the MoSCoW-prioritised product backlog of 41 user stories across 10 features, and the planning of four sprints over two releases, together with the schedule and the architecture and technical choices that frame the realisation. The following chapters describe the development of the two releases.

Chapter 3

Release 1: Foundations & Core Marketplace

Plan

Introduction

This chapter covers Release 1, *Foundations & Core Marketplace*, which establishes the identity layer, the seller-onboarding loop and the core selling capabilities the rest of the platform builds on. After a release overview, use-case diagram, domain model and backlog, it presents the release's two sprints: **Sprint 1 (Authentication & Seller Onboarding)** and **Sprint 2 (Store & Product Management)**.

3.1 Release 1 Overview

Release 1 delivers the marketplace skeleton over two sprints. Sprint 1 builds authentication and the seller-onboarding loop, so that a visitor can register, verify their email, sign in, and request the seller role for an administrator to review. Sprint 2 builds store and product management, so that an approved seller can open a store, customise its storefront, populate a catalogue and run a store-wide sale, while customers browse that catalogue by category. The architecture and the technology stack that frame this realisation were established in Section 1.4 and are not repeated here; this release instantiates them for the foundational features.

3.1.1 Release 1 Use-Case Diagram

Figure 3.1 scopes the use cases delivered by Release 1 around its four actors. A *Visitor* can register and browse; a *Customer* (the role granted on registration) can additionally authenticate and apply to become a seller; a *Seller* inherits every customer ability and gains store and product management; the *Administrator* reviews seller applications. The release deliberately excludes transactions, AI verification and the premium tier, which belong to Release 2.

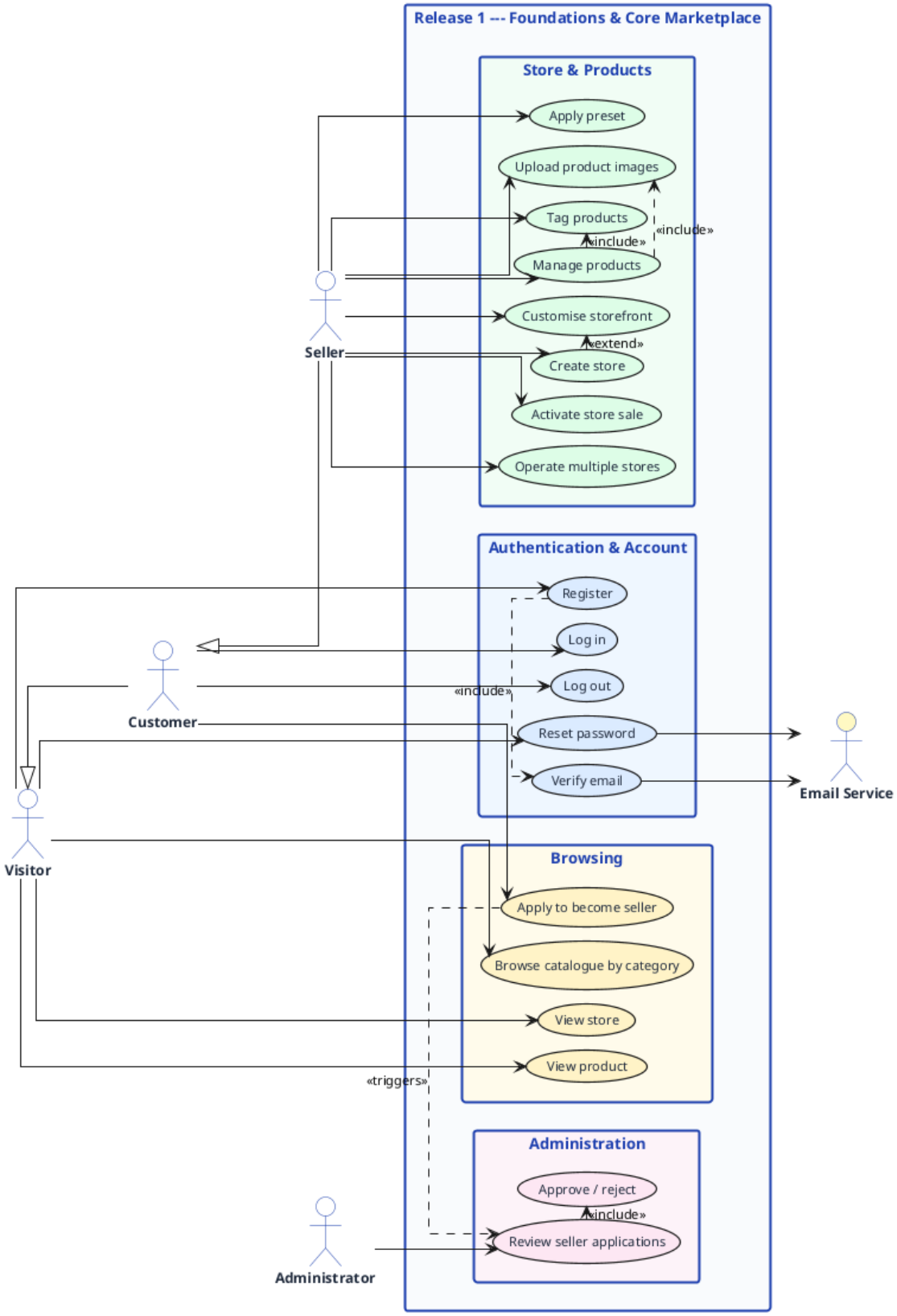

Figure 3.1: Release 1 use-case diagram (foundations and core marketplace).

3.1.2 Release 1 Domain Model

Figure 3.2 shows the slice of the domain model introduced by Release 1: the `User` together with its `Sessions` and optional `SellerApplication`, the `Store` it owns, and the `Product` catalogue with its

images, tags and category hierarchy. This subset of the global class diagram of Section 1.4 is the data foundation on which the two sprints operate.

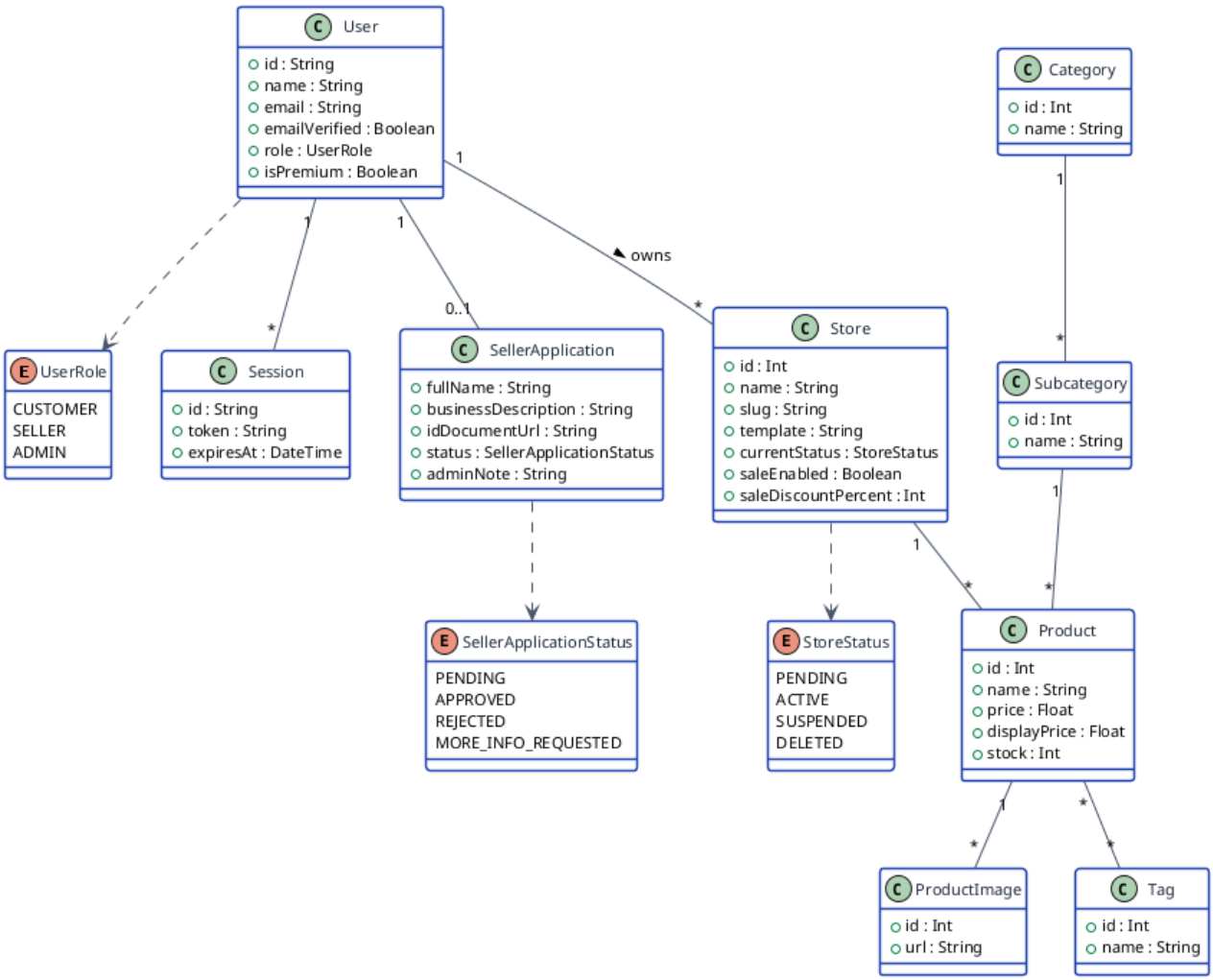

Figure 3.2: Release 1 domain model (subset of the platform class diagram).

3.2 Release 1 Backlog

The release backlog draws sixteen user stories from the product backlog of Section 1.4 and allocates them to the two sprints, as summarised in Table 3.1. Each sprint carries an estimated duration, in line with the overall five-to-six-month schedule.

Table 3.1: Release 1 backlog, user stories grouped by sprint and feature.

Sprint	Feature	User stories	Est. time
Sprint 1	Authentication	US01 account creation, US02 email verification, US03 sign-in, US04 sign-out, US37 password reset	~1 month
	Administration	US18 seller-application review	
Sprint 2	Store Management	US05 store creation, US06 readable store slug, US07 storefront WYSIWYG editor, US08 style presets, US09 multiple stores (premium), US34 store-wide sales	~1.5 months
	Product Management	US10 product CRUD, US11 product images, US12 product tags, US13 category browsing	
		Release 1 total	**~2.5 months**

3.3 Sprint 1, Authentication & Seller Onboarding

3.3.1 Sprint 1 Objective

Sprint 1 delivers the identity layer of the marketplace and the loop that turns a customer into a seller. By the end of the sprint a visitor can create an account secured by email verification, sign in and out through database-backed sessions, recover access through an email-based password reset, and submit a seller application; an administrator can review that application and, on approval, promote the account to the seller role. Authentication is provided by the **Better-Auth** library, integrated into the NestJS backend through a thin adapter; it ships email/password credentials, email verification, database-persisted sessions and cookie transport, so that no cryptography is hand-written. Every protected route is gated by an authentication guard and the `@Roles()` decorator, with object-level ownership enforced in the service layer.

3.3.2 Sprint 1 Use-Case Diagram

Figure 3.3 details the sprint's use cases. Registration *includes* email verification; submitting a seller application *triggers* an administrative review, which may *extend* into an approval, a rejection or a request for more information.

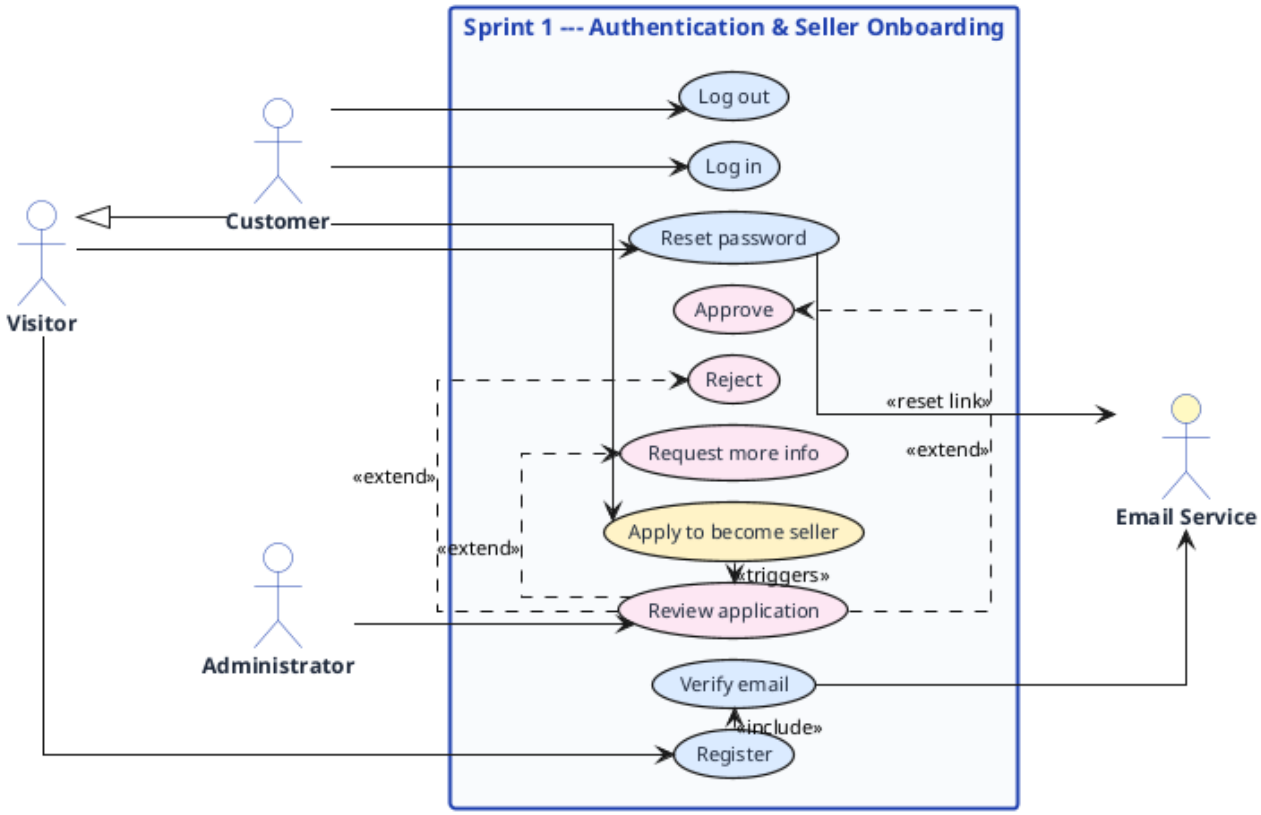

Figure 3.3: Sprint 1 use-case diagram (authentication and seller onboarding).

3.3.3 Analysis and Design

The five central use cases of the sprint are specified textually below, then the registration control flow is given as an activity diagram and the authentication exchange as a sequence diagram.

3.3.3.1 Textual Use-Case Descriptions

Table 3.2: Textual description of the *Register* use case.

Use case: Register	
Actor	Visitor (unauthenticated).
Objective	Create a marketplace account with an email address and a password.
Pre-conditions	No account exists for the supplied email address.
Post-conditions	A `User` is created with the CUSTOMER role and `emailVerified = false`; a one-time verification token is generated and emailed; a pending-verification session cookie is returned.
Nominal scenario	1. The visitor opens the sign-up page. 2. The visitor enters a name, email and password. 3. The visitor submits the form. 4. Better-Auth creates the user and persists the credentials. 5. The system sends the verification email and shows a "check your inbox" screen.
Alternative scenarios	A1. Email already registered: an error invites the visitor to sign in instead. E1. Weak or malformed input: client- and server-side validation reject the request and the form is redisplayed with messages.

Table 3.3: Textual description of the *Log in* use case.

Use case: Log in	
Actor	Registered user (Customer, Seller or Administrator).
Objective	Open an authenticated session and reach the role-appropriate dashboard.
Pre-conditions	The account exists.
Post-conditions	A database-backed session token is issued in an HTTP-only cookie; the user is redirected to their dashboard.
Nominal scenario	1. The user opens the sign-in page. 2. The user enters their email and password. 3. Better-Auth validates the credentials against the `User` table. 4. A session row is created and the session cookie is set. 5. The user is redirected to their dashboard.
Alternative scenarios	A1. Invalid credentials: a generic error is shown without revealing which field failed. A2. Unverified email: sign-in succeeds but store creation and ordering remain blocked until the email is verified.

Table 3.4: Textual description of the *Reset password* use case.

Use case: Reset password	
Actor	Registered user who has forgotten their password (unauthenticated); Email Service (secondary).
Objective	Regain access to the account by setting a new password through an emailed link.
Pre-conditions	The user cannot sign in and has access to the email address on the account.
Post-conditions	A one-time, time-limited reset token is generated and emailed; once it is consumed the password is replaced and every existing session for that account is revoked.
Nominal scenario	1. The user opens the "forgot password" page from the sign-in screen and enters their email. 2. Better-Auth generates a one-time token and the system emails a reset link. 3. The user opens the link and enters a new password with confirmation. 4. Better-Auth validates the token, replaces the password and revokes all other sessions. 5. The user is returned to the sign-in page to log in with the new password.
Alternative scenarios	A1. Unknown email: the same generic confirmation is shown, so the request never reveals whether an account exists. E1. Expired or invalid token: the reset is refused and the user is invited to request a new link.

Table 3.5: Textual description of the *Apply to become seller* use case.

Use case: Apply to become seller	
Actor	Customer (with a verified email).
Objective	Request the seller role by submitting a seller application.
Pre-conditions	The user is an authenticated customer and has no application in progress.
Post-conditions	A `SellerApplication` is persisted with status PENDING and queued for administrator review.
Nominal scenario	1. The customer opens the "become a seller" form. 2. The customer enters their full name, phone number, home address and business description, optionally a website, and uploads an identity document. 3. The customer submits the form. 4. The system stores the application as PENDING and notifies the administrators.
Alternative scenarios	A1. An application already exists: its current status is displayed instead of the form. E1. A required document is missing: validation rejects the submission.

Table 3.6: Textual description of the *Review seller application* use case.

Use case: Review seller application	
Actor	Administrator.
Objective	Decide on a pending seller application.
Pre-conditions	At least one application is in the PENDING or MORE_INFO_REQUESTED state.
Post-conditions	The application status becomes APPROVED, REJECTED or MORE_INFO_REQUESTED; on approval the applicant's role is promoted to SELLER, unlocking store creation; an optional administrator note is recorded.
Nominal scenario	1. The administrator opens the applications panel. 2. The administrator selects a pending application and inspects its details and documents. 3. The administrator approves, rejects, or requests more information, optionally adding a note. 4. The system updates the status and notifies the applicant.
Alternative scenarios	A1. Request more information: the status becomes MORE_INFO_REQUESTED and the applicant may amend and resubmit.

3.3.3.2 Activity Diagram

Figure 3.4 models the registration and email-verification control flow. Two decision points govern the path: a uniqueness check on the email at sign-up, and a validity check on the one-time token at verification. Only once the token is accepted and `emailVerified` is set to `true` does the account unlock the protected actions of store creation and order placement.

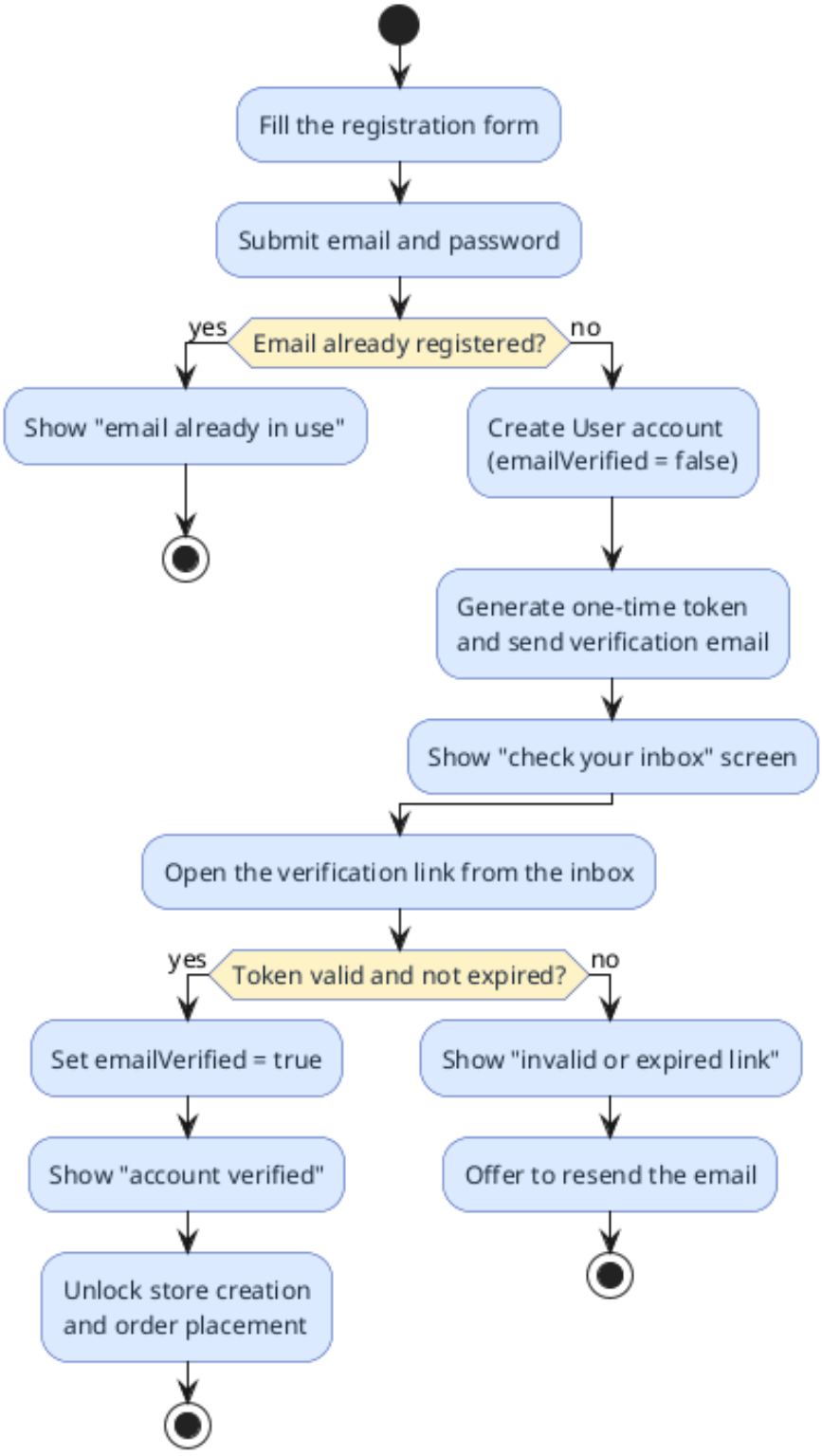

Figure 3.4: Activity diagram, registration and email verification.

3.3.3.3 Sequence Diagram

Figure 3.5 traces the three-stage authentication exchange between the React single-page application, the NestJS API, the Better-Auth adapter, the mail service and the database. Registration creates the user and sends the verification email; verification flips `emailVerified` to `true`; login validates the

credentials and issues a fresh database-backed session whose token travels in an HTTP-only cookie and is checked by the authentication guard on every subsequent protected request.

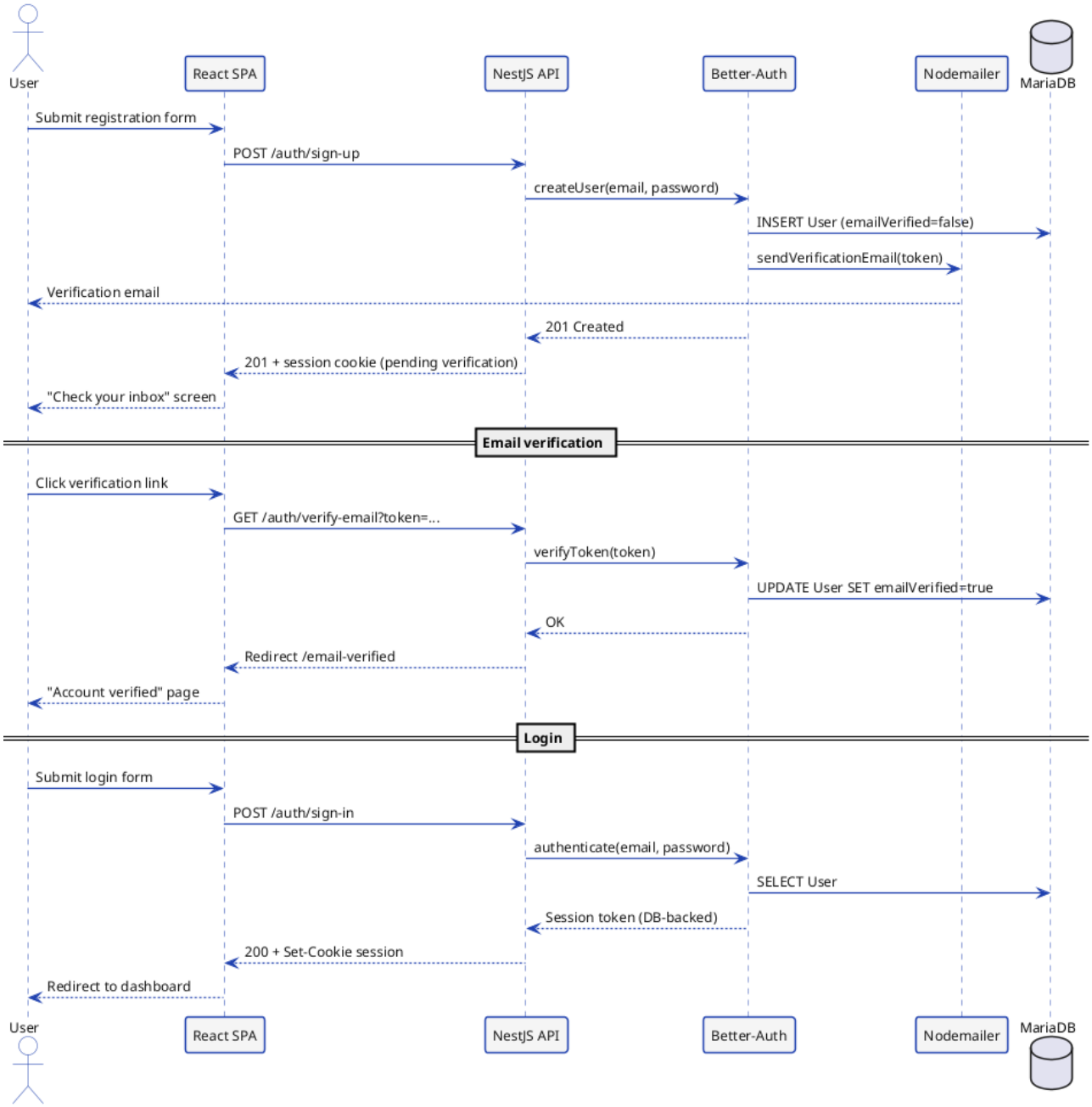

Figure 3.5: Sequence diagram, registration, email verification and login.

3.3.4 Realisation

The sprint delivers the authentication and onboarding interfaces shown in the following figures. Each frame is a placeholder for the corresponding screenshot of the running application.

[Screenshot to insert: Registration page, name, email and password form]

Figure 3.6: Registration page.

[Screenshot to insert: Email-verification screen, "check your inbox" and verified confirmation]

Figure 3.7: Email-verification screen.

[Screenshot to insert: Login page, credentials form and session redirect]

Figure 3.8: Login page.

[Screenshot to insert: Password-reset flow, "forgot password" request page and the new-password page reached from the emailed link]

Figure 3.9: Password-reset pages.

[Screenshot to insert: Seller-application form, identity, business details and document upload]

Figure 3.10: Seller-application form.

[Screenshot to insert: Administrator review panel, pending applications with approve / reject / request-more-info actions]

Figure 3.11: Administrator seller-application review panel.

3.4 Sprint 2, Store & Product Management

3.4.1 Sprint 2 Objective

Sprint 2 turns an approved seller into an active merchant. By the end of the sprint a seller can create a store reachable at a human-readable slug, customise its storefront across seven sections through a WYSIWYG editor, populate a product catalogue with images and tags, and run a store-wide sale; customers can browse that catalogue by category. The storefront editor is built with **Craft.js** [19], where clicking any section on the live canvas opens its settings panel, with undo/redo and preset starting points. Several capabilities exposed here are **premium-tiered**: the full customisation set, the operation of multiple stores (US09) and the AI-assisted storefront redesign (US35) are built on the storefront foundation in this sprint, but their premium *entitlement* is delivered by the premium tier in Release 2 (Chapter 3.4.4); until then free accounts use the curated subset and a single store.

3.4.2 Sprint 2 Use-Case Diagram

Figure 3.12 details the sprint's use cases. Storefront customisation *includes* applying a preset and uploading a banner, and *extends* into the AI redesign; product management *includes* image upload and tagging.

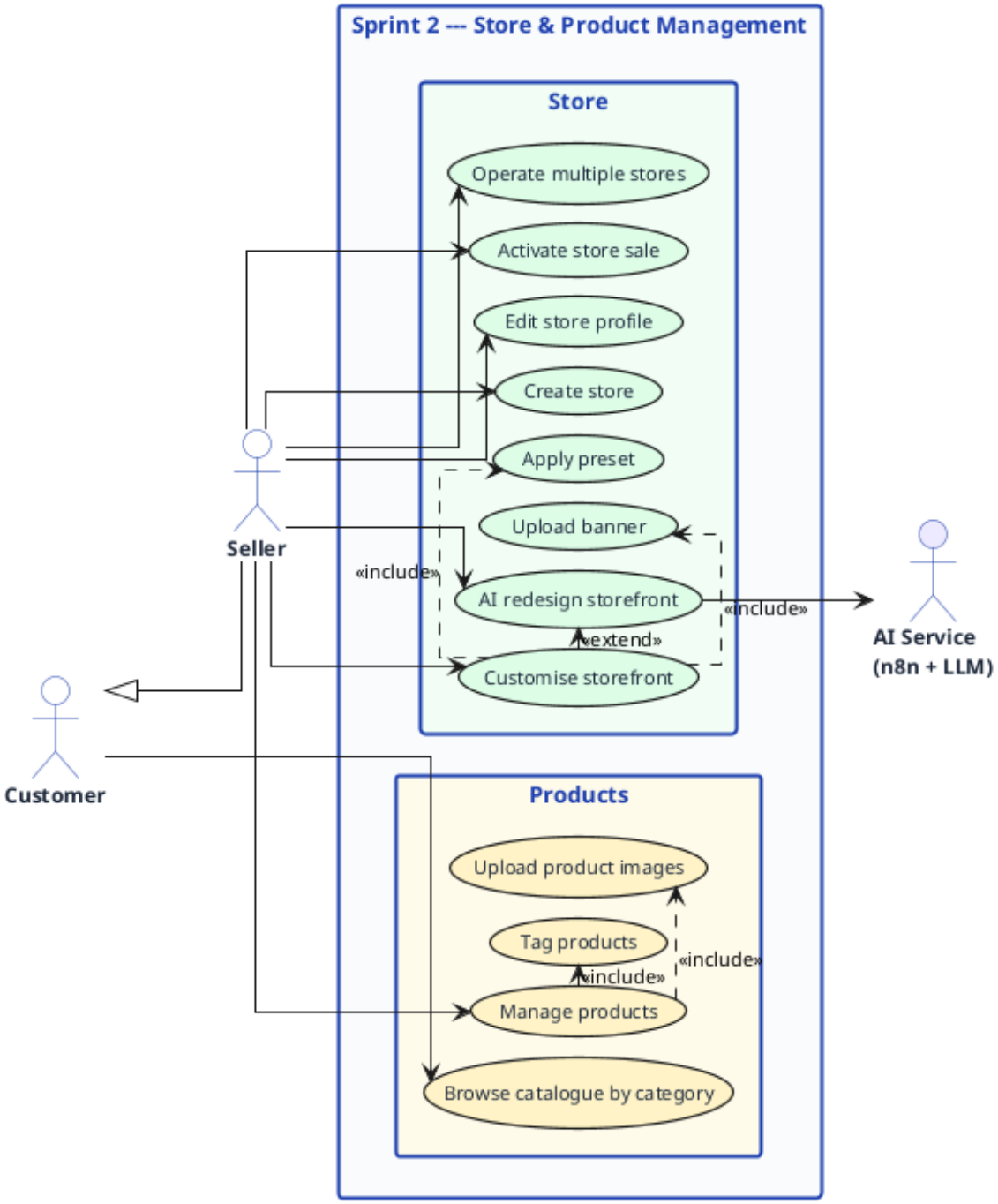

Figure 3.12: Sprint 2 use-case diagram (store and product management).

3.4.3 Analysis and Design

The four central use cases of the sprint are specified textually, then the store-creation and AI-redesign control flows are given as activity diagrams and the AI-redesign exchange as a sequence diagram.

3.4.3.1 Textual Use-Case Descriptions

Table 3.7: Textual description of the *Create store* use case.

Use case: Create store	
Actor	Seller.
Objective	Open a store with a name, description and logo, reachable at a readable URL.
Pre-conditions	The user holds the seller role; a free-tier seller owns no store yet.
Post-conditions	A `Store` is created with status PENDING and a unique slug; a `StoreStatusLog` entry is written; the store is queued for AI verification; the storefront editor opens on the default preset.
Nominal scenario	1. The seller opens the create-store form. 2. The seller enters a name, description and logo. 3. The seller submits the form. 4. The system derives a slug from the name and enforces uniqueness, appending a numeric suffix on collision. 5. The store is persisted as PENDING, the status is logged and AI verification is queued. 6. The storefront editor opens.
Alternative scenarios	A1. A free-tier seller already owns a store: the seller is prompted to upgrade to premium. E1. Unsupported logo type or size: validation rejects the upload.

Table 3.8: Textual description of the *Customise storefront* use case.

Use case: Customise storefront	
Actor	Seller.
Objective	Style the storefront across its seven sections through the WYSIWYG editor.
Pre-conditions	The seller owns the store.
Post-conditions	The `StoreCustomization` object is updated and persisted; premium-only fields are applied only for premium accounts.
Nominal scenario	1. The seller opens the Craft.js editor. 2. The seller clicks a section (Theme, Announcement Bar, Banner, Header, Product Grid, Sidebar or Footer) to open its settings. 3. The seller adjusts colours, fonts and radius with a live preview and undo/redo. 4. The seller optionally applies a preset (*Default* or *Minimal*; *Modern* or *Classic* for premium). 5. The seller saves the customisation.
Alternative scenarios	A1. A free account edits a premium-only field: the control is locked with an upgrade hint. A2. Unsaved changes on exit: the seller is warned before discarding.

Table 3.9: Textual description of the *AI redesign storefront* use case.

Use case: AI redesign storefront	
Actor	Seller (premium for the gated capability); AI Service (secondary).
Objective	Regenerate the full storefront from a natural-language prompt.
Pre-conditions	The seller owns the store and is in the editor.
Post-conditions	A complete `StoreCustomization` covering all seven sections is generated, sanitised, persisted and applied to the live canvas for review.
Nominal scenario	1. The seller clicks "AI Redesign" and enters a prompt. 2. The backend checks ownership and forwards the prompt with the store name and description to the n8n workflow. 3. n8n builds the system prompt, calls the Groq model and parses and sanitises the returned JSON. 4. The design is persisted and applied to the canvas. 5. The seller reviews, refines or saves.
Alternative scenarios	E1. The model returns invalid JSON: the sanitiser rejects it, the current design is kept and an error is shown.

Table 3.10: Textual description of the *Create product* use case.

Use case: Create product	
Actor	Seller.
Objective	Add a product with images and tags to the store catalogue.
Pre-conditions	The seller owns a store.
Post-conditions	A `Product` is persisted with `verificationStatus = pending` and `aiReviewed = false`; its images are stored with one set as the thumbnail; its tags are linked; the product is queued for AI verification.
Nominal scenario	1. The seller opens product management. 2. The seller enters a name, description, price, stock and sub-category. 3. The seller uploads one or more images, validated by MIME type and size. 4. The seller assigns tags. 5. The seller saves; the product is queued for AI verification.
Alternative scenarios	A1. The seller edits or deletes an existing product. E1. An unsupported or oversized image is rejected.

3.4.3.2 Activity Diagrams

Figure 3.13 models store creation, including slug derivation with collision handling and the transition to a PENDING store queued for verification. Figure 3.14 models the AI-redesign flow, from the prompt to the sanitised design applied on the live canvas, with the seller's review-and-save decision.

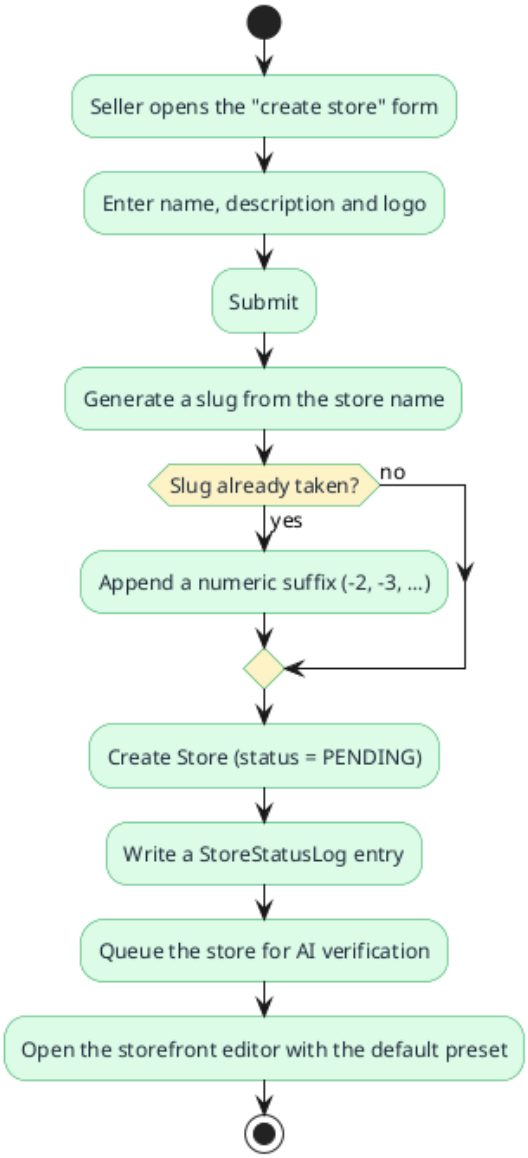

Figure 3.13: Activity diagram, store creation.

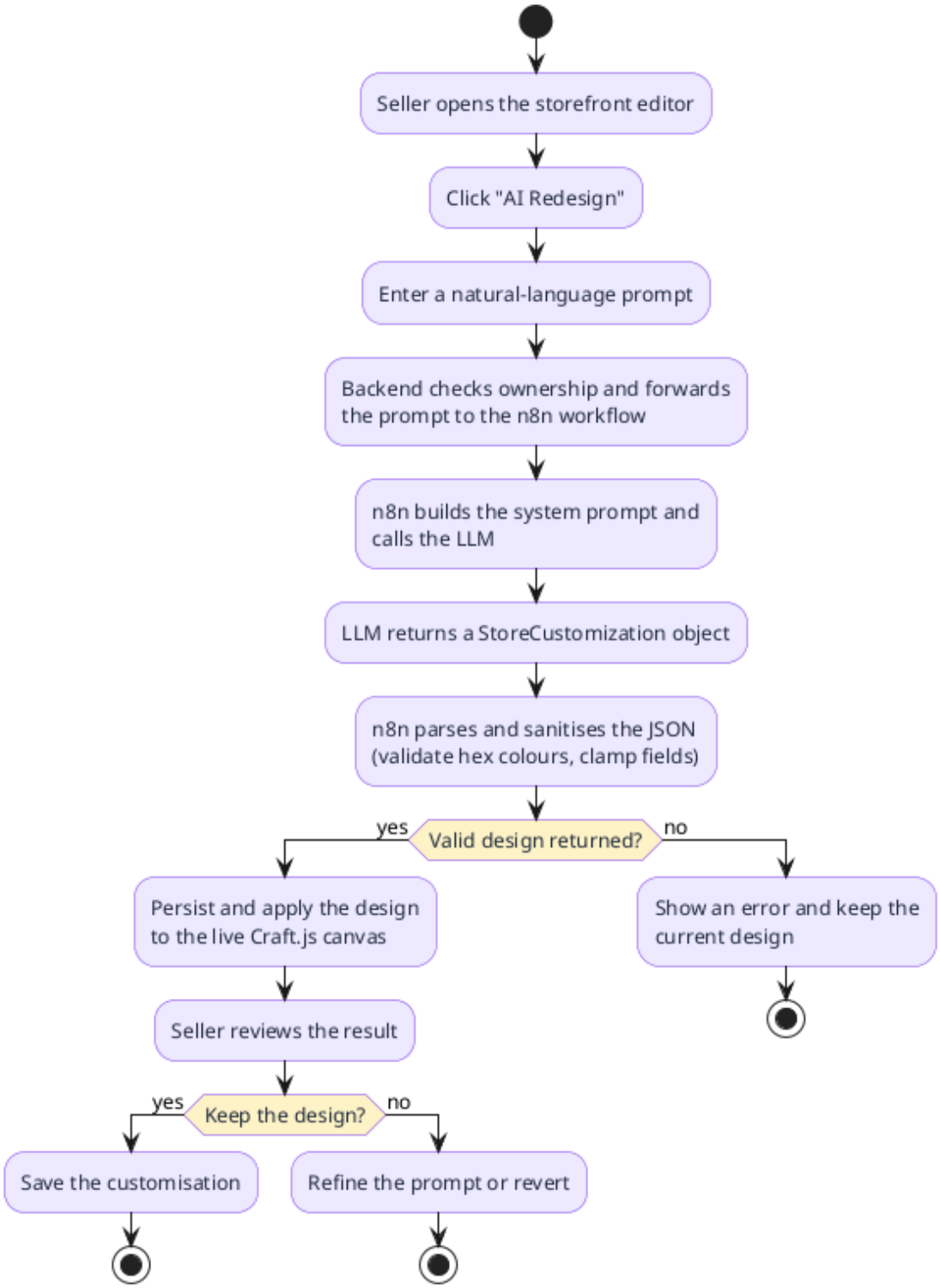

Figure 3.14: Activity diagram, AI storefront redesign.

3.4.3.3 Sequence Diagram

Figure 3.15 traces the AI-redesign exchange. The seller's prompt travels from the editor to the NestJS API, which forwards it, after an ownership check, to the n8n webhook; n8n builds the prompt, calls the Groq model, then parses and sanitises the returned `StoreCustomization` before the backend persists it and streams it back to the live editor.

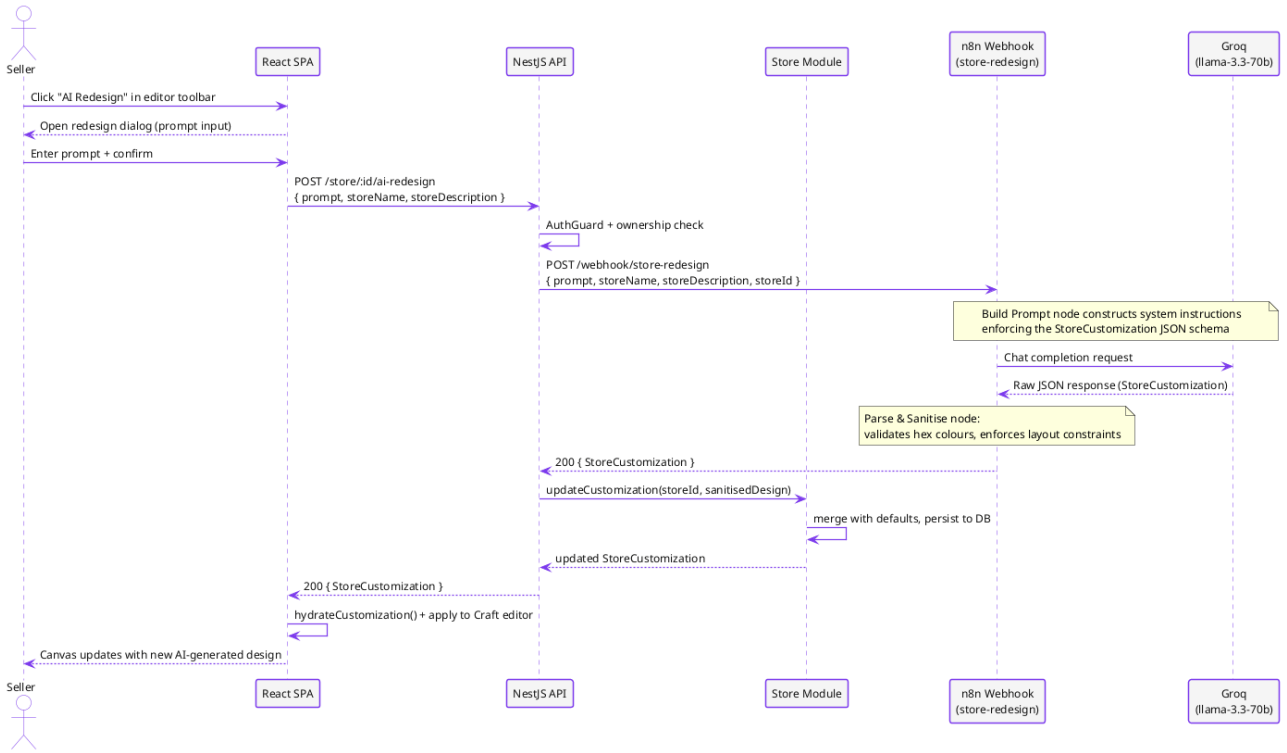

Figure 3.15: Sequence diagram, AI storefront redesign.

3.4.4 Realisation

The sprint delivers the store and product interfaces shown below. Each frame is a placeholder for the corresponding screenshot of the running application.

[Screenshot to insert: Seller store dashboard, metrics, store status and quick actions]

Figure 3.16: Seller store dashboard.

[Screenshot to insert: Storefront WYSIWYG editor, Craft.js canvas with the seven-section settings panel]

Figure 3.17: Storefront customisation editor.

[Screenshot to insert: AI redesign dialog, natural-language prompt input]

Figure 3.18: AI storefront-redesign dialog.

[Screenshot to insert: Product management, catalogue list with create/edit form, image upload and tags]

Figure 3.19: Product management interface.

[Screenshot to insert: Store-wide sale configuration, discount percentage, label and expiry]

Figure 3.20: Store-wide sale configuration.

Conclusion

Release 1 laid the foundations of the marketplace. Sprint 1 delivered the identity layer, registration with email verification, sessions, password reset and role-based access, plus the seller-onboarding loop. Sprint 2 turned approved sellers into active merchants, store creation, the storefront editor, AI-assisted redesign, product management and store-wide sales, and opened catalogue browsing to customers. The marketplace can now be populated, but not yet transact; Release 2 adds that.

Chapter 4

Release 2: Transactions, Intelligence & Delivery

Plan

Introduction

This chapter covers Release 2, *Transactions, Intelligence & Delivery*, which turns the populated marketplace into a transacting, self-moderating and deployable product. After a release overview, use-case diagram, domain model and backlog, it presents the release's two sprints: **Sprint 3 (Orders, Payment & Administration)** delivers the cart, multi-store checkout, Stripe payment and administration tooling; **Sprint 4 (AI Verification & Premium)** adds the AI verification pipeline, the premium tier, and the testing and deployment that bring the platform to production.

4.1 Release 2 Overview

Release 2 completes the marketplace over two sprints. Sprint 3 makes the catalogue transactable: customers fill a cart, check out across multiple stores in a single order, and pay through Stripe, while administrators moderate stores and products, curate featured content and manage categories. Sprint 4 adds the intelligence and delivery layers: automated AI verification of new stores and products, the AI database-chat assistant, live cross-store product suggestions on the product page, the premium subscription tier, a light/dark theming option, and the test suite and deployment pipeline. As before, the architecture and technology stack of Section 1.4 are not repeated; this release instantiates them for the transactional, intelligence and delivery features.

4.1.1 Release 2 Use-Case Diagram

Figure 4.1 scopes the use cases delivered by Release 2. Customers gain the cart, checkout and payment; sellers gain order visibility, the premium tier and AI redesign; administrators gain moderation, featured curation, category management and the authority to override AI verdicts; the AI service and the Stripe payment gateway act as secondary actors.

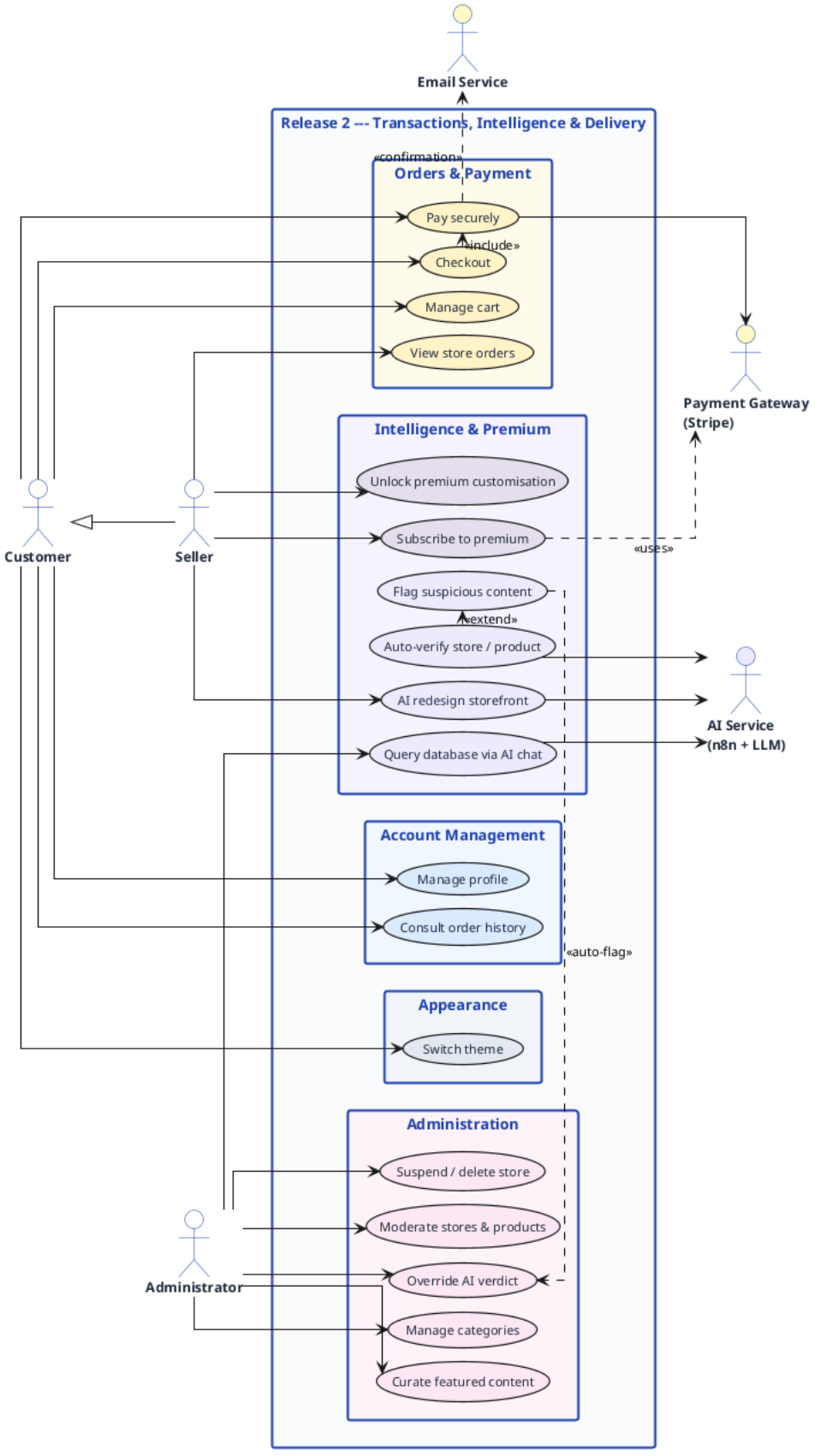

Figure 4.1: Release 2 use-case diagram (transactions, intelligence and delivery).

4.1.2 Release 2 Domain Model

Figure 4.2 shows the slice of the domain model exercised by Release 2: the `Order` and its `OrderItem`s, the payment fields stamped on the order, the verification flags on `Store` and `Product`, the `StoreStatusLog` audit trail and the featured-content entities.

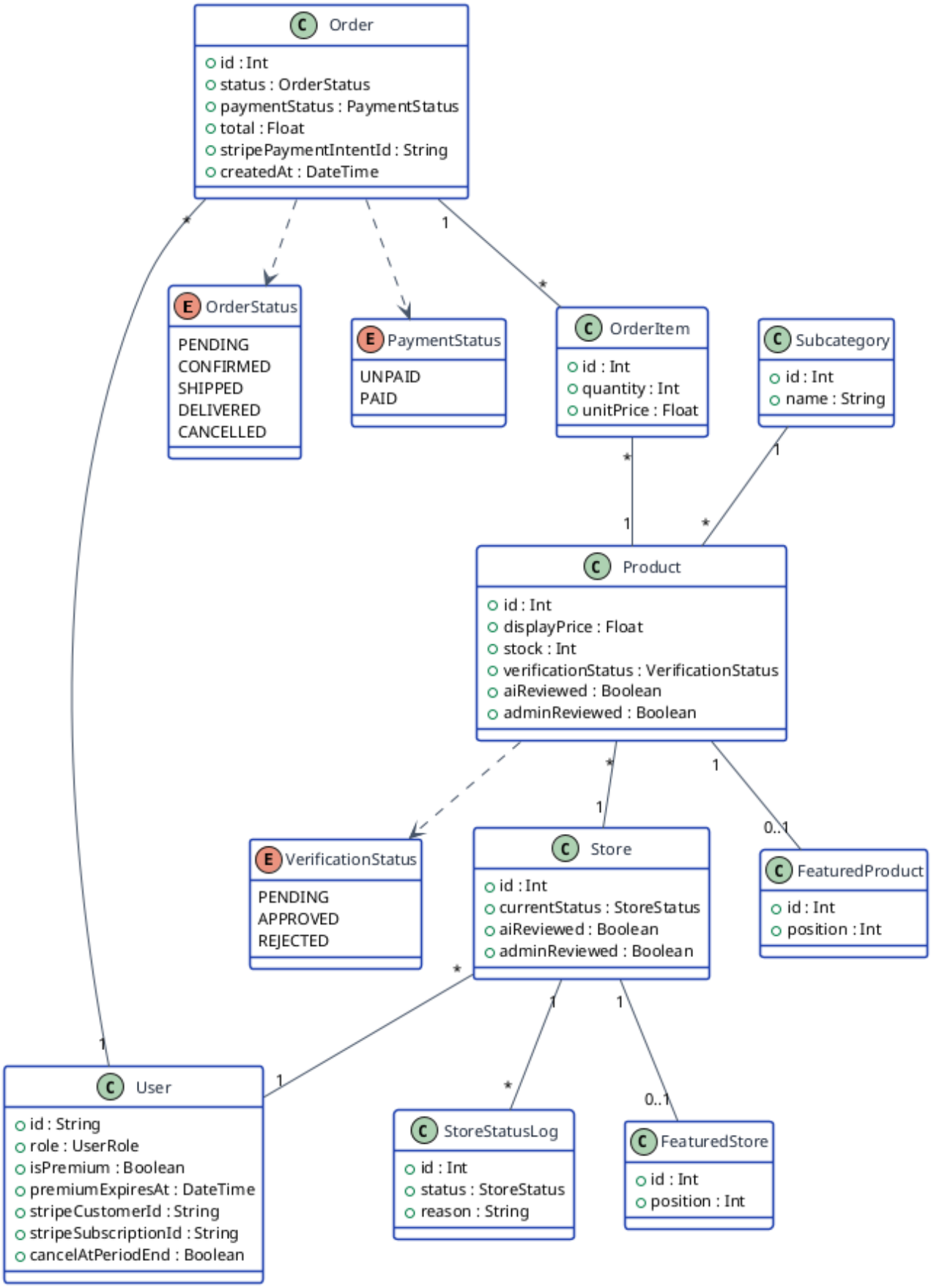

Figure 4.2: Release 2 domain model (subset of the platform class diagram).

4.2 Release 2 Backlog

The release backlog draws the remaining user stories from the product backlog of Section 1.4, including the payment story US36, the account-management stories US38–US39 and the theming story US40, and allocates them to the two sprints, as summarised in Table 4.1.

Table 4.1: Release 2 backlog, user stories grouped by sprint and feature.

Sprint	Feature	User stories	Est. time
Sprint 3	Order System & Payment	US14 shopping cart, US15 multi-store checkout, US16 order-confirmation email, US17 seller order view, US36 secure card payment	~1.5 months
	Account Management	US38 profile management, US39 purchase history	
	Administration	US19 store suspension/deletion, US20 featured-content curation, US21 category management	
Sprint 4	AI Verification	US22 AI store/product verification, US23 AI verdict override, US24 verified badge, US35 AI storefront redesign	~1.5 months
	Product Suggestions	US41 cross-store product suggestions	
	Premium Tier	US25 premium subscription, US26 premium customisation controls, US27 premium preservation on expiry	
	Appearance & Theming	US40 light/dark theming	
	CI/CD & Deployment	US28 service containerisation, US29 CI image builds, US30 Terraform provisioning, US31 SSH deployment, US32 CI-managed secrets, US33 DNS automation	
		Release 2 total	**~3 months**

4.3 Sprint 3, Orders, Payment & Administration

4.3.1 Sprint 3 Objective

Sprint 3 makes the marketplace transactable and governable. By the end of the sprint a customer can build a cart, check out across several stores in a single order, and pay securely through Stripe, receiving a confirmation email; a seller can see the orders that include their products; every signed-in user can manage their own account, viewing and editing their profile, changing their password and reviewing their purchase history, and an administrator can moderate stores and products, suspend or delete a store, curate featured content and manage the category tree. The payment flow is the centrepiece:

amounts are computed server-side, charged through a single Stripe [20] PaymentIntent, and confirmed asynchronously by a signature-verified webhook.

4.3.2 Sprint 3 Use-Case Diagram

Figure 4.3 details the sprint's use cases. Checkout *includes* the secure payment, which in turn drives the Stripe gateway and the confirmation email; suspending a store is an *extension* of store moderation.

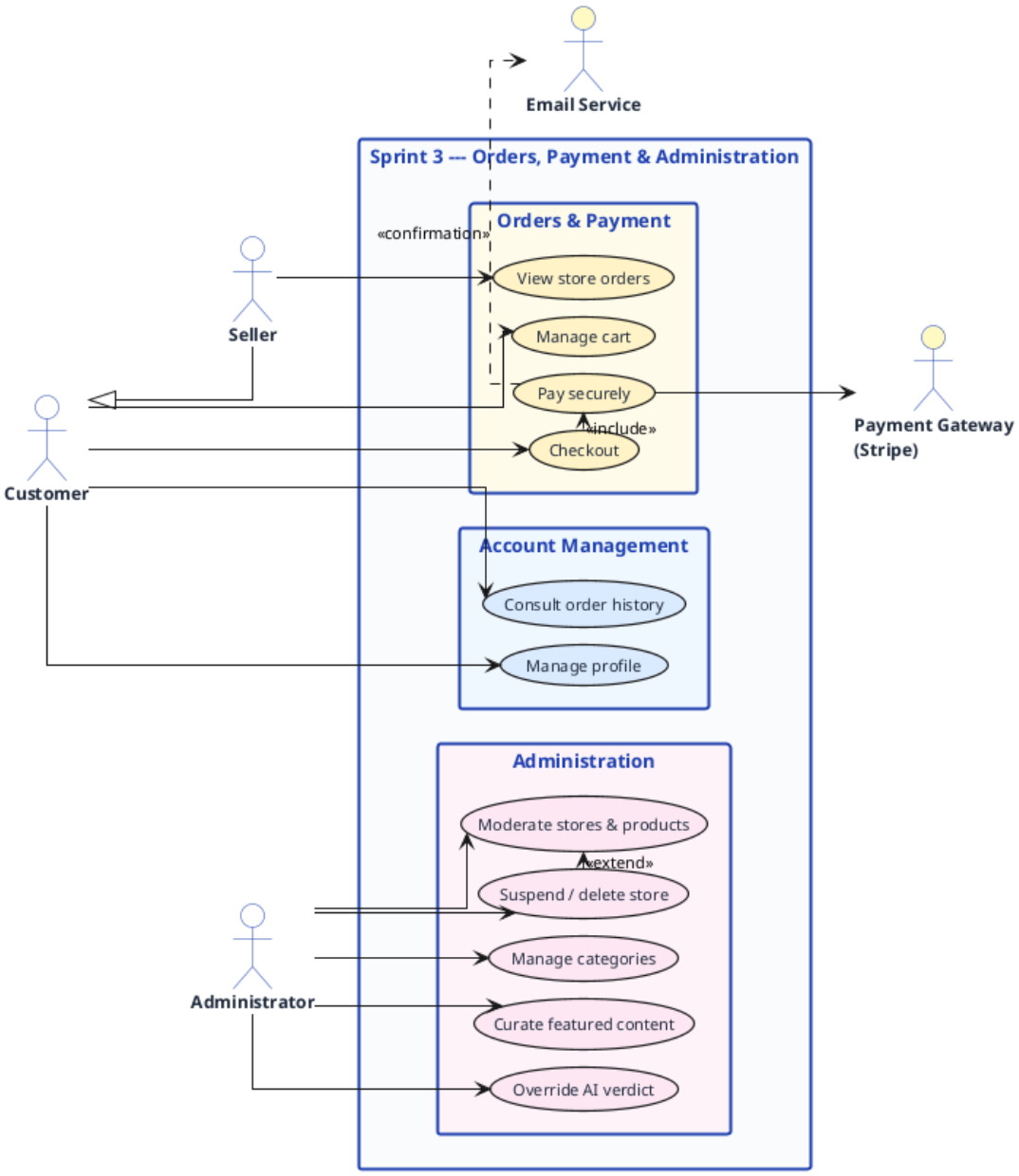

Figure 4.3: Sprint 3 use-case diagram (orders, payment and administration).

4.3.3 Analysis and Design

The five central use cases of the sprint are specified textually, then the checkout-and-payment control flow is given as an activity diagram and the order placement as a sequence diagram. The Stripe payment mechanism is then described in its own section.

4.3.3.1 Textual Use-Case Descriptions

Table 4.2: Textual description of the *Checkout* use case.

Use case: Checkout	
Actor	Customer (authenticated, verified).
Objective	Convert the cart into an order, even when it spans several stores.
Pre-conditions	The cart is non-empty and the customer is signed in with a verified email.
Post-conditions	A single PENDING `Order` is created with one `OrderItem` per product line across all stores, and stock is reserved.
Nominal scenario	1. The customer opens the checkout page. 2. The customer reviews the items, quantities and shipping form. 3. The customer confirms. 4. The server prices each line server-side (using the sale price when a sale is active), reserves stock, and creates one order spanning all stores with its order items. 5. The payment step opens.
Alternative scenarios	A1. Insufficient stock on a line: the line is flagged and checkout halts. A2. Unverified email: checkout is blocked until verification.

Table 4.3: Textual description of the *Pay securely* use case.

Use case: Pay securely	
Actor	Customer; Payment Gateway (Stripe, secondary).
Objective	Pay the order total securely and have the order confirmed.
Pre-conditions	A PENDING order with a positive total and reserved stock exists.
Post-conditions	A Stripe PaymentIntent is opened for the server-computed total; on a signature-verified success webhook the order is marked PAID and CONFIRMED and a confirmation email is sent; on failure the reserved stock is released.
Nominal scenario	1. The server computes the grand total and converts it from TND to USD minor units. 2. The server opens one PaymentIntent for the total with an idempotency key and stamps its id on the order. 3. The client confirms the payment with Stripe. 4. Stripe sends a signed webhook; the backend verifies the signature against the raw body. 5. On `payment_intent.succeeded` the order becomes PAID and CONFIRMED and the confirmation email is sent.
Alternative scenarios	E1. Payment fails or is canceled: the order is marked failed and the reserved stock is released (idempotently). E2. Lost webhook: a periodic reconciliation sweep retrieves the intent from Stripe and settles or restocks the order.

Table 4.4: Textual description of the *Manage profile* use case.

Use case: Manage profile	
Actor	Any authenticated user (Customer, Seller or Administrator).
Objective	View and update one's own account: display name, phone and avatar, change the password, and review the purchase history.
Pre-conditions	The user is signed in.
Post-conditions	The editable fields (name, phone, avatar) are updated; a password change revokes the other sessions; privileged fields (role, premium status, email) are never altered through this surface.
Nominal scenario	1. The user opens their account page (a customer through the tabbed account page, a seller through the dashboard). 2. The user edits the name, phone or avatar and saves; the server accepts only those fields. 3. The user optionally changes the password by supplying the current and a new one. 4. The user opens the purchase-history view to consult past orders, paginated.
Alternative scenarios	A1. A privileged field is submitted: the server ignores it, so a user can never self-grant a role or premium status. E1. Invalid input (e.g. a too-short new password or a mismatched confirmation): validation rejects the change with a message.

Table 4.5: Textual description of the *Suspend store* use case.

Use case: Suspend store	
Actor	Administrator.
Objective	Suspend or delete a store that violates the marketplace rules.
Pre-conditions	The target store exists.
Post-conditions	The store `currentStatus` becomes SUSPENDED (or DELETED, cascade-removing its products); a `StoreStatusLog` entry records the change; the store leaves public browsing.
Nominal scenario	1. The administrator opens the store moderation panel. 2. The administrator selects a store and inspects it. 3. The administrator chooses suspend or delete, optionally with a reason. 4. The system updates the status, writes a `StoreStatusLog` entry, and on delete cascades to the store's products.
Alternative scenarios	A1. Reactivate a suspended store: `currentStatus` returns to ACTIVE.

Table 4.6: Textual description of the *Override AI verdict* use case.

Use case: Override AI verdict	
Actor	Administrator.
Objective	Override the automated verification verdict on a store or product.
Pre-conditions	The entity carries an AI verdict (`aiReviewed = true`) or has been flagged.
Post-conditions	The entity's `adminReviewed` flag is set and its verification status reflects the administrator's decision, which takes precedence over the AI verdict.
Nominal scenario	1. The administrator opens the moderation panel. 2. The administrator reviews the AI verdict and rationale on the flagged entity. 3. The administrator approves or rejects, overriding the AI. 4. The system sets `adminReviewed` and updates the verification status; the administrator's decision is final.
Alternative scenarios	A1. The administrator agrees with the AI: no override is needed and the AI verdict stands.

4.3.3.2 Activity Diagram

Figure 4.4 models the checkout-and-payment control flow. After stock is reserved and the pending order priced server-side, a single Stripe PaymentIntent is opened; the signed webhook then drives the order either to PAID/CONFIRMED on success or back to released stock on failure.

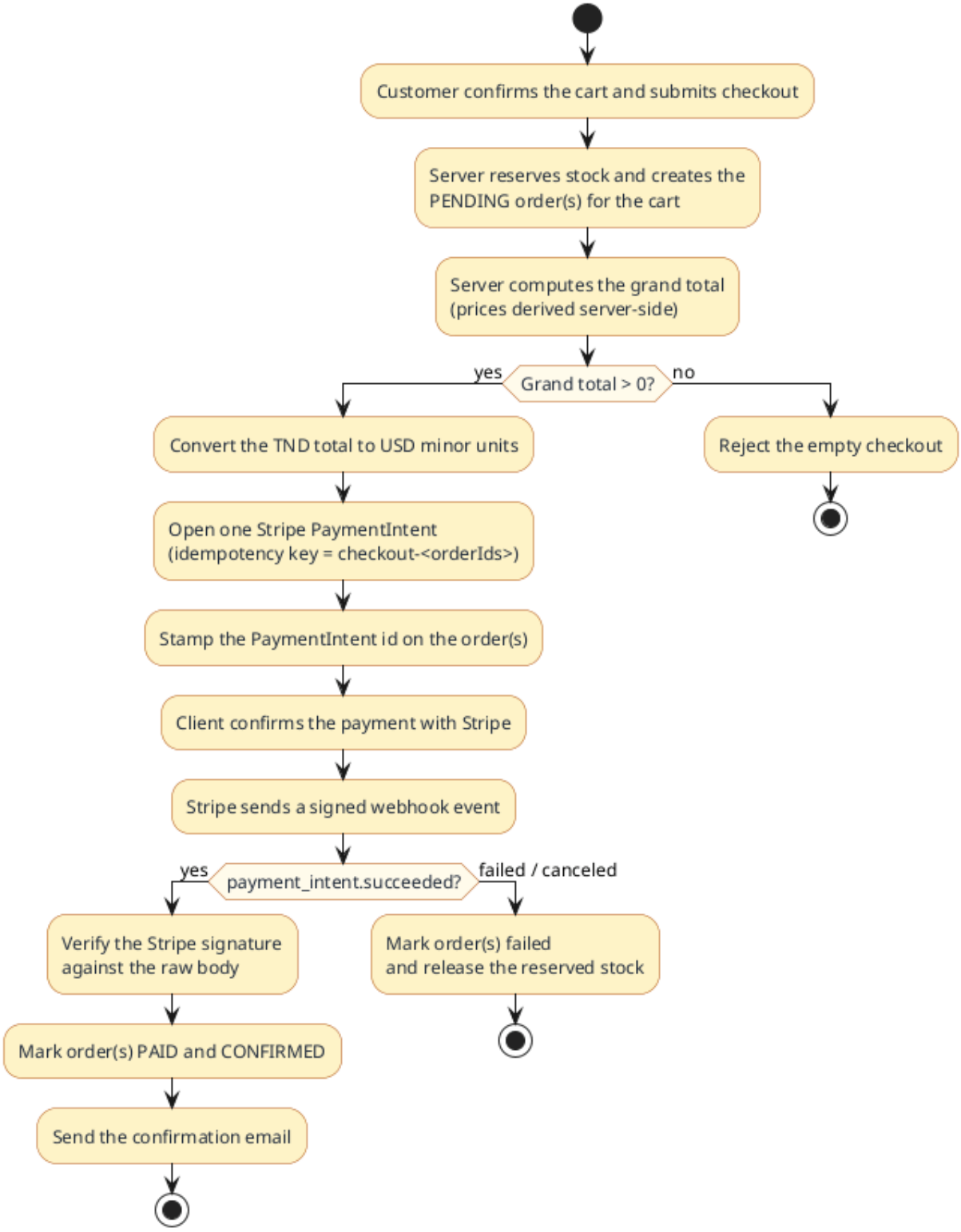

Figure 4.4: Activity diagram, multi-store checkout and Stripe payment.

4.3.3.3 Sequence Diagram

Figure 4.5 traces order placement across the React application, the NestJS API and the database, including the creation of the single multi-store order and its items.

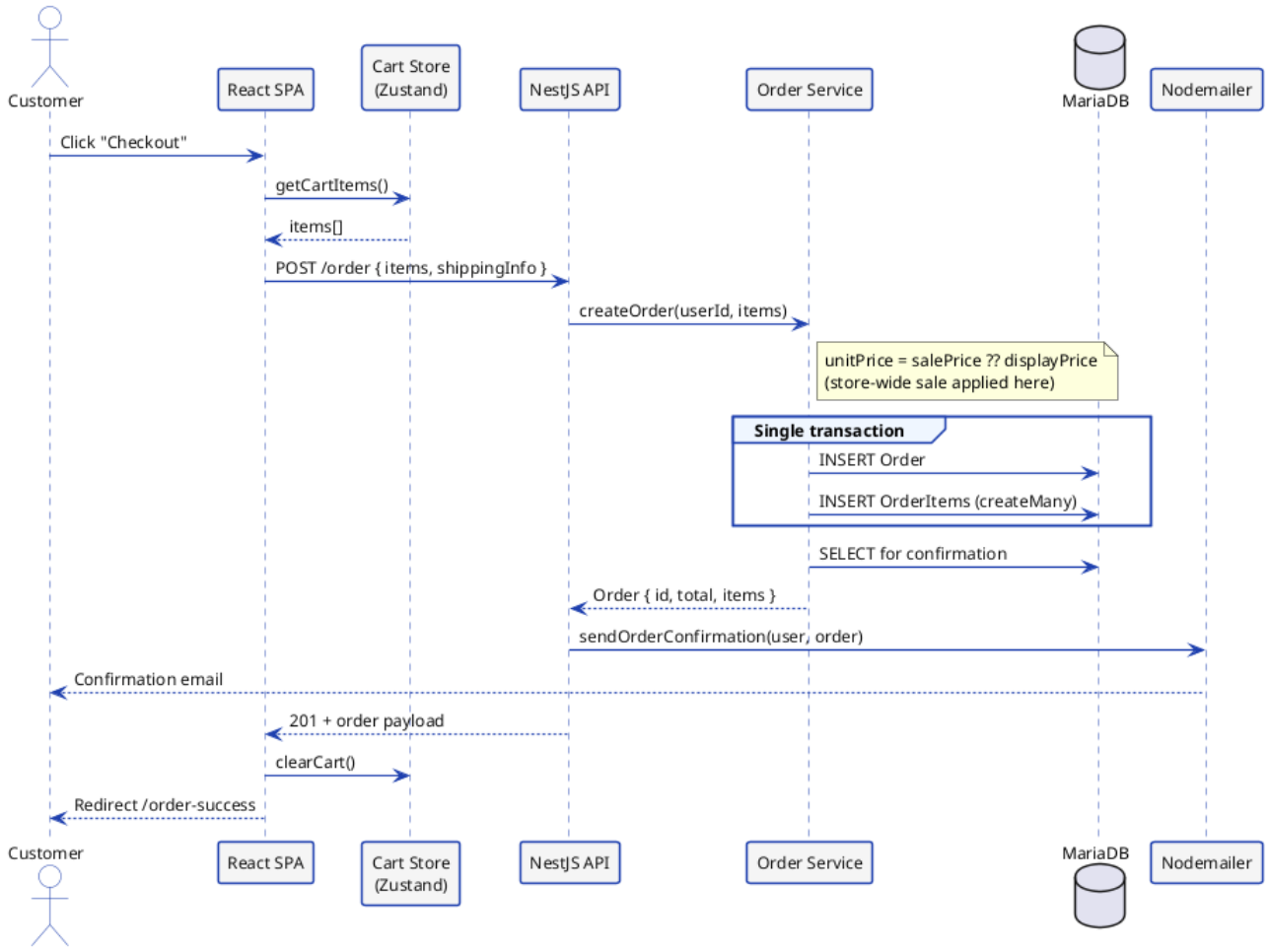

Figure 4.5: Sequence diagram, multi-store order placement.

4.3.4 Stripe Payment Design

Payment is the most security-sensitive flow of the platform, and its design follows a single principle: **the client never dictates what it is charged**. When the customer checks out, the backend reserves stock, creates the pending order(s) for the cart, and computes the grand total from the server-side prices. Stripe accounts cannot settle in Tunisian dinar, so the dinar total is converted to United States dollar minor units through a configurable rate (`TND_TO_USD_RATE`) while the interface continues to display dinar. A single PaymentIntent is then opened for the whole cart, even when it spans several stores, with an **idempotency key** derived from the order identifiers, so that a retried request never creates a duplicate charge. The PaymentIntent identifier is stamped on the order(s) for later reconciliation. Listing 4.1 shows the core of this server-side charge.

```
const intent = await stripe.paymentIntents.create(
  {
    // amount is computed server-side and converted TND -> USD minor units
    amount: tndToStripeUsdMinor(grandTotal),
    currency: 'usd',
```

```
    metadata: {
      orderIds: orderIds.join(','),
      displayCurrency: 'TND',
      displayAmount: grandTotal.toFixed(3),
    },
  },
  { idempotencyKey: `checkout-${orderIds.join('-')}` },
);
await orderService.setPaymentIntent(orderIds, intent.id);
```

Listing 4.1: Server-side PaymentIntent creation (amount derived on the server).

Confirmation is asynchronous. Stripe calls an **unauthenticated** `POST /payments/webhook` endpoint, it is deliberately public, because the caller is Stripe rather than a logged-in user. Trust is established not by a session but by **verifying the Stripe signature against the raw request body**; an unverifiable payload is rejected. On a `payment_intent.succeeded` event the order(s) are marked PAID and CONFIRMED and the confirmation email is sent; on a failed or canceled event the reserved stock is released. The handler is idempotent, so a duplicated webhook is harmless. Finally, a periodic cleanup sweep reconciles checkouts left unpaid past a staleness window: it retrieves each PaymentIntent from Stripe and either settles a paid-but-unacknowledged order or releases the stock of an abandoned one, so a lost webhook can never strand stock or a payment.

4.3.5 Realisation

The sprint delivers the transaction and administration interfaces shown below. Each frame is a placeholder for the corresponding screenshot of the running application.

[Screenshot to insert: Checkout page, multi-store cart summary, shipping form and Stripe payment element]

Figure 4.6: Checkout page with Stripe payment.

[Screenshot to insert: Account page, profile view/edit (name, phone, avatar), change-password form and the paginated purchase history]

Figure 4.7: User account and profile page.

[Screenshot to insert: Administrator dashboard, marketplace metrics and navigation]

Figure 4.8: Administrator dashboard.

[Screenshot to insert: Moderation panel, stores and products with suspend / delete and verdict-override actions]

Figure 4.9: Store and product moderation panel.

[Screenshot to insert: Category management, category and sub-category tree]

Figure 4.10: Category management.

[Screenshot to insert: Featured-content curation, selecting featured stores and products for the landing page]

Figure 4.11: Featured-content curation.

4.4 Sprint 4, AI Verification & Premium Tier

4.4.1 Sprint 4 Objective

Sprint 4 adds the intelligence and monetisation layers. Every new store and product is screened automatically by an AI verification pipeline before it becomes visible; suspicious content is flagged for the administrator, who remains the final authority. Administrators gain a natural-language database chat assistant for diagnostics; buyers see live cross-store product suggestions, a cheaper same-item offer and relevant alternatives, on every product page; and sellers , and only sellers, can take out a *recurring monthly* premium subscription through Stripe Checkout, managed afterwards through the Stripe Customer Portal, that unlocks the full customisation set and multiple stores; this is a new payment mechanism, distinct from the one-time charge of Sprint 3. The AI capabilities are delivered through four independent n8n workflows invoked by the backend over HTTP. Finally, the sprint rounds out the user experience with an appearance option: a light and a dimmed dark theme, chosen explicitly or following the operating-system preference and applied consistently across the application chrome through CSS design tokens.

4.4.2 Sprint 4 Use-Case Diagram

Figure 4.12 details the sprint's use cases. Automatic verification *includes* returning a verdict with a rationale and may *extend* into flagging content for the administrator to override; subscribing to premium *unlocks* the advanced customisation and multiple stores through the Stripe gateway, and the subscription is afterwards managed through that same gateway.

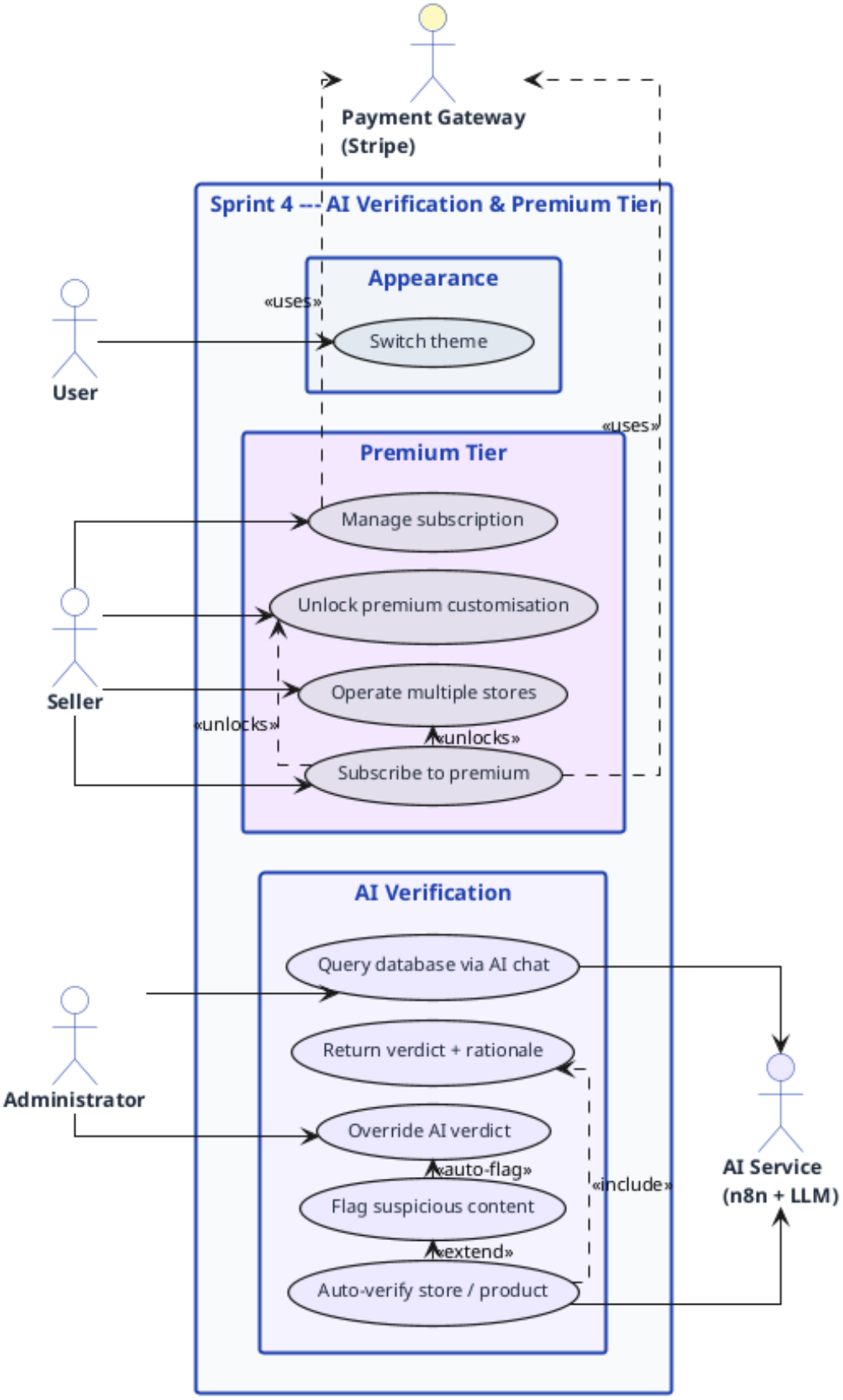

Figure 4.12: Sprint 4 use-case diagram (AI verification and premium tier).

4.4.3 Analysis and Design

The five central use cases of the sprint are specified textually, then the verification and moderation control flows are given as activity diagrams, and the verification and product-suggestion exchanges as sequence diagrams.

4.4.3.1 Textual Use-Case Descriptions

Table 4.7: Textual description of the *AI verification* use case.

Use case: AI verification	
Actor	AI Service (n8n + vision LLM); triggered by store or product creation.
Objective	Automatically screen every new store and product before it becomes visible.
Pre-conditions	A new entity exists with `aiReviewed = false` and `verificationStatus = pending`, and its images are uploaded.
Post-conditions	`aiReviewed` becomes `true` and the verification status is set to APPROVED (auto-published) or the entity is flagged for administrator review; a verdict and rationale are recorded.
Nominal scenario	1. After image upload, the backend fires the `ai-verify` webhook with the listing text and image list. 2. n8n downloads each image and submits it with the text to the vision model. 3. The per-image verdicts are aggregated, flagging the entity if any image is flagged. 4. The decision is posted to the `/ai-review` callback. 5. The backend publishes clean entities and queues flagged ones for moderation.
Alternative scenarios	E1. The webhook or model fails: the entity stays PENDING and is surfaced in the administrator's queue for manual review.

Table 4.8: Textual description of the *AI chat query* use case.

Use case: AI chat query	
Actor	Administrator; AI Service (n8n + LLM + SQL tool).
Objective	Query the database in natural language for diagnostics and insight.
Pre-conditions	The administrator is authenticated.
Post-conditions	A natural-language answer, derived from a generated SQL query executed read-only against the database, is returned; no data is modified.
Nominal scenario	1. The administrator types a question in the chat assistant. 2. The backend forwards it to the n8n chat workflow. 3. The AI agent translates the question into SQL, executes it against MariaDB through the SQL tool, and formats the result. 4. The answer is returned to the administrator.
Alternative scenarios	A1. An ambiguous question: the assistant asks for clarification, using its conversation memory for context.

Table 4.9: Textual description of the *Subscribe to / manage premium subscription* use case.

Use case: Subscribe to / manage premium subscription	
Actor	Seller; Payment Gateway (Stripe, secondary).
Objective	Take out and manage a recurring monthly premium subscription that unlocks full storefront customisation and multiple stores.
Pre-conditions	The user holds the seller role.
Post-conditions	Once Stripe confirms the subscription, `isPremium` becomes `true` and `premiumExpiresAt` is set to the current billing period end; the entitlement renews automatically, and premium customisation and multi-store operation are unlocked.
Nominal scenario	1. The seller opens the upgrade page and starts the subscription. 2. The backend opens a Stripe Checkout session in subscription mode and redirects the seller to the hosted page. 3. The seller pays on Stripe. 4. On a signature-verified `customer.subscription.created` webhook the account is marked premium until the current period end. 5. Each renewal fires `customer.subscription.updated` and advances the expiry. 6. The seller later manages the subscription, cancel, update card, download invoices, through the Stripe Customer Portal.
Alternative scenarios	A1. Cancellation through the portal is *cancel-at-period-end*: premium is kept until the paid period ends, then a `customer.subscription.deleted` webhook revokes it. A2. A non-seller is redirected to the become-a-seller flow. A3. An administrator grants premium manually, a comp account with no Stripe subscription. A4. After expiry the premium customisation values are preserved and stay visible to buyers; only further editing of premium fields is locked (US27).

Table 4.10: Textual description of the *Switch theme* use case.

Use case: Switch theme	
Actor	Any user, whether authenticated or not.
Objective	Choose how the application is rendered: a light theme, a dimmed dark theme, or automatically following the operating-system preference.
Pre-conditions	None; the theme switch is reachable from any page through the navigation chrome.
Post-conditions	The interface re-themes immediately and the choice is persisted on the device, so it is re-applied before first paint on later visits.
Nominal scenario	1. The user opens the theme switch and picks light, dark or system. 2. The interface re-themes at once through CSS design tokens, with no page reload. 3. The preference is stored locally and restored on the next visit before the first paint, so there is no flash of the wrong theme.
Alternative scenarios	A1. The user picks *system*: the interface follows the OS light/dark setting and updates live when that setting changes. A2. A seller storefront keeps its own seller-chosen palette and is intentionally not overridden by the application theme.

Table 4.11: Textual description of the *View product suggestions* use case.

Use case: View product suggestions	
Actor	Customer (visitor or authenticated); AI suggestion service (secondary).
Objective	On a product page, surface a cheaper same-item offer and relevant alternatives drawn from across the marketplace.
Pre-conditions	The viewed product exists and is publicly visible.
Post-conditions	The page shows a validated "same item, cheaper" offer (or none) and a list of eligible alternatives; only approved, available, in-stock products other than the viewed one are ever shown.
Nominal scenario	1. The customer opens a product page. 2. The frontend requests suggestions for that product on its own asynchronous query, so the page paints immediately. 3. The backend calls the suggestion workflow, which reasons over the catalogue and returns candidate ids. 4. The backend hydrates and validates the ids, dropping ineligible ones and keeping the better deal only if genuinely cheaper. 5. The validated suggestions render in their own section once ready.
Alternative scenarios	E1. The workflow errors, times out or returns nothing usable: the backend falls back to a deterministic catalogue query (same category, ranked by name similarity and tag overlap), so the section is never empty on a transient failure. A1. Nothing qualifies even from the fallback: the section stays empty rather than showing irrelevant items.

4.4.3.2 Activity Diagrams

Figure 4.13 models the AI verification pipeline, including the per-image screening loop inside the n8n workflow and the branch that either publishes a clean entity or routes a flagged one to moderation. Figure 4.14 models the administrator's content-moderation decision over a flagged entity.

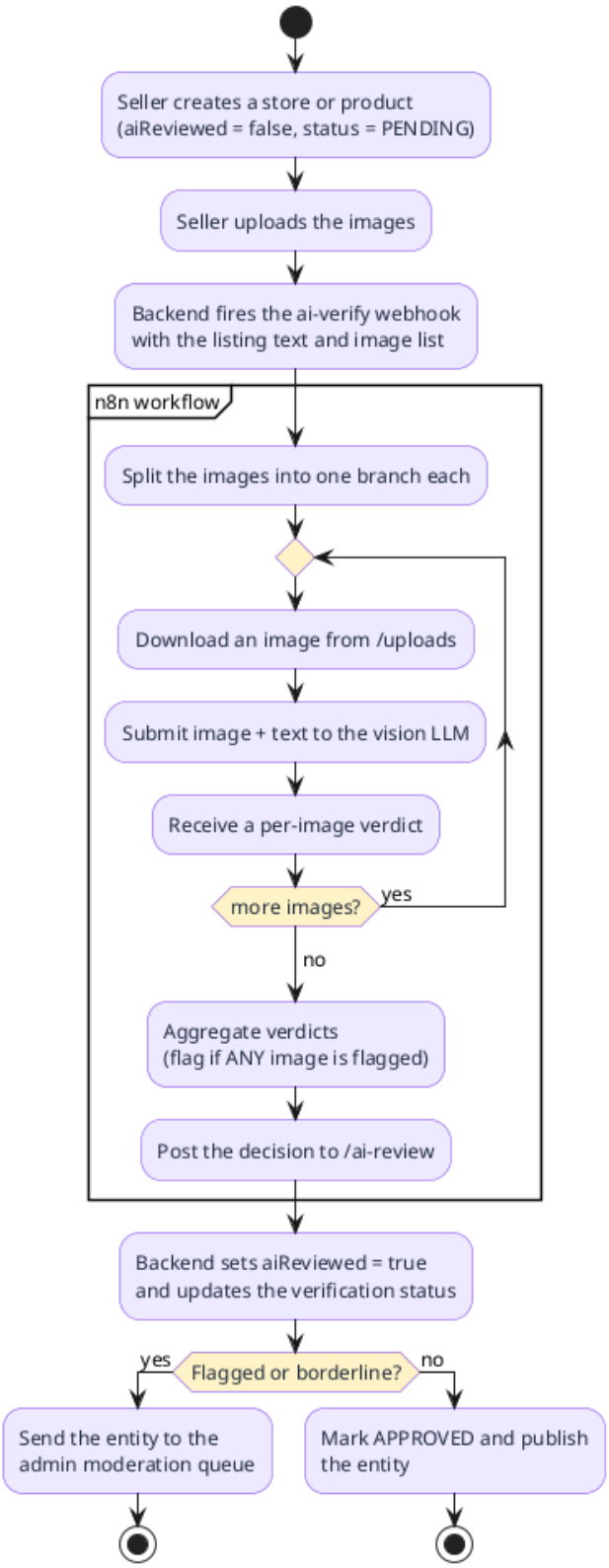

Figure 4.13: Activity diagram, AI verification pipeline.

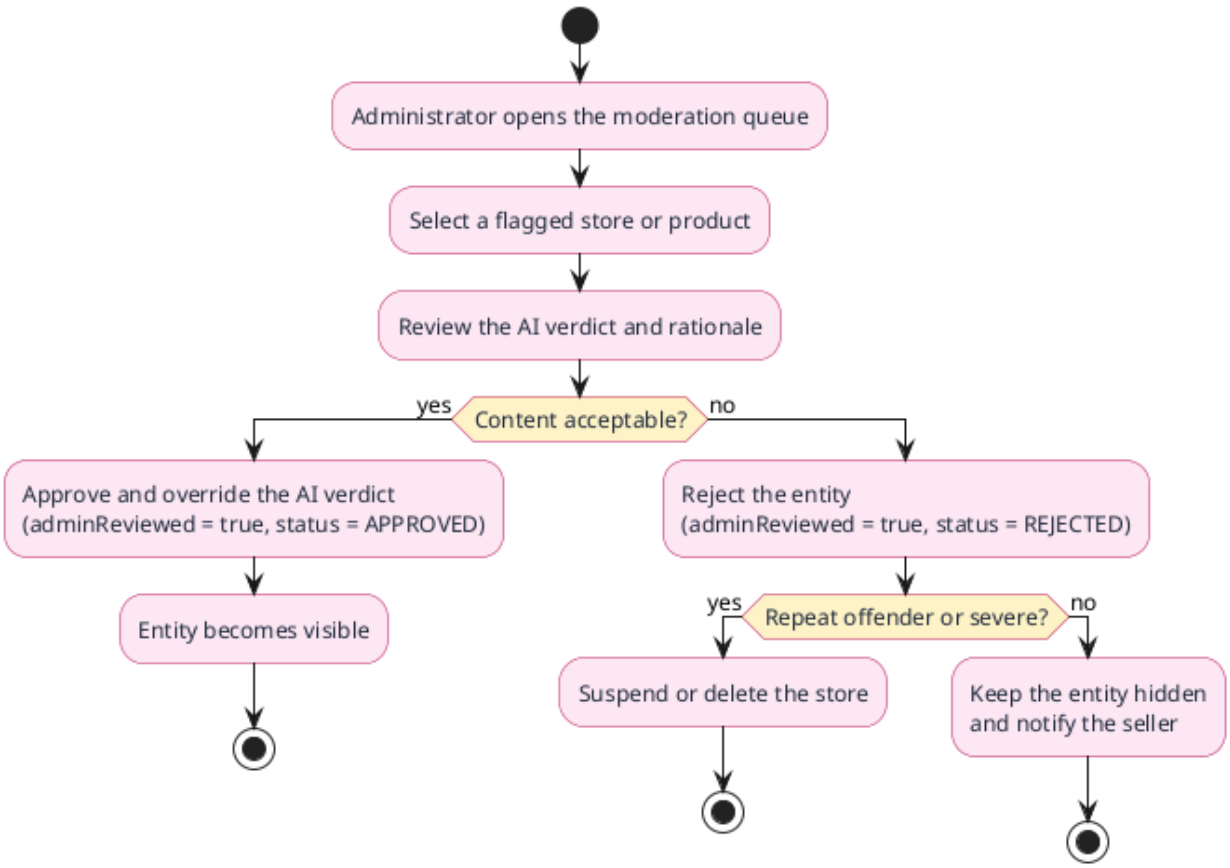

Figure 4.14: Activity diagram, content moderation.

4.4.3.3 Sequence Diagram

Figure 4.15 traces the verification exchange for both products and stores: creation with a pending status, image upload, the per-image moderation loop against the vision model, the aggregated verdict posted to the callback, and the alternative branches of automatic approval or administrative override.

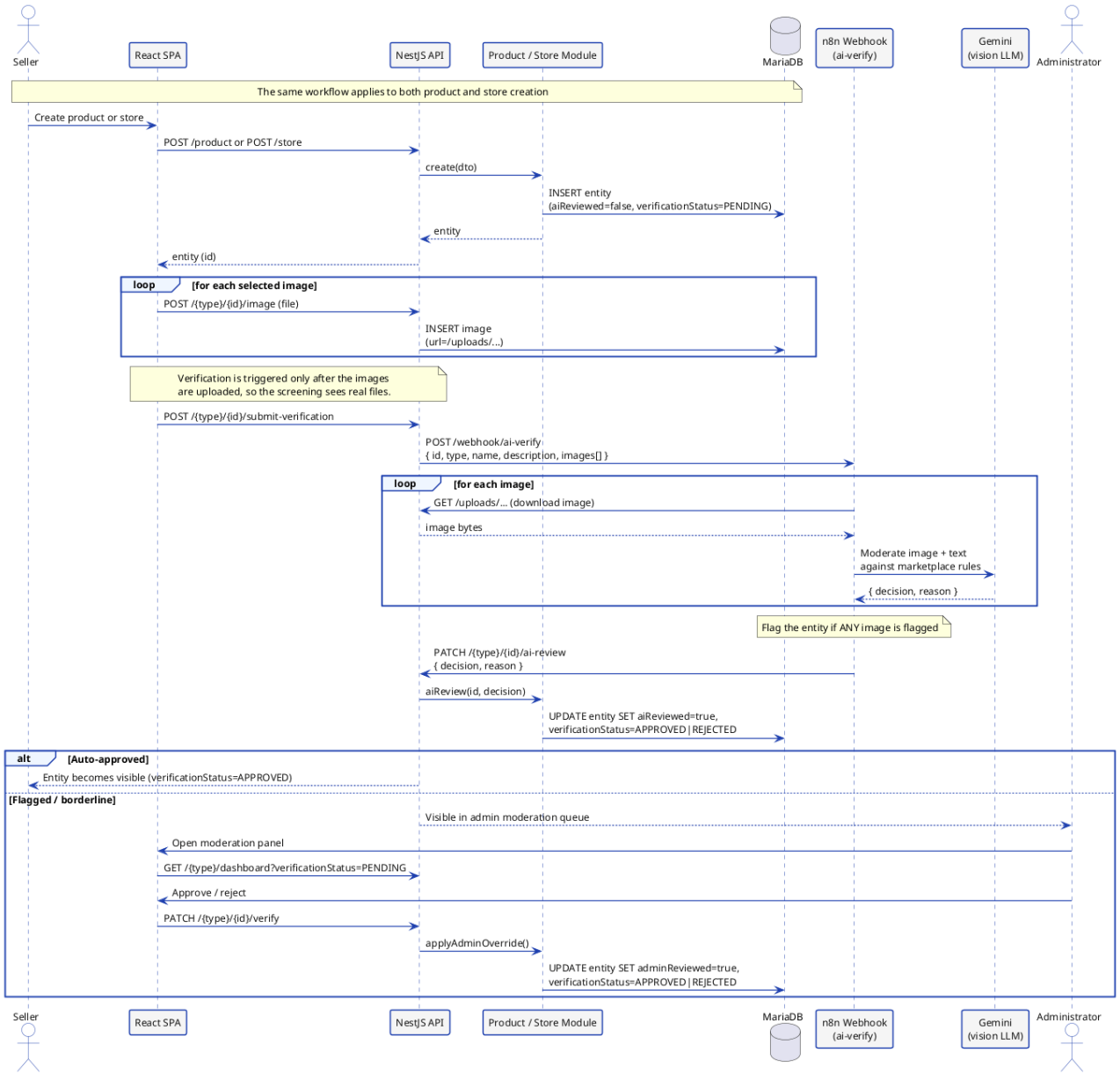

Figure 4.15: Sequence diagram, AI verification with administrative override.

4.4.4 n8n Workflow Design

The platform's AI features are carried by four independent n8n workflows deployed on the same engine, each exposed as an HTTP webhook the backend invokes. The **verification** workflow (Figure 4.16) fans the image list out into one branch per image, downloads each file, moderates it against the marketplace rules with a vision model, and posts the aggregated verdict back to the callback. The **database chat** workflow (Figure 4.17) wraps an AI agent, with a chat model, conversation memory and a SQL execution tool, that answers an administrator's natural-language question by querying the database. The **storefront-redesign** workflow (Figure 4.18), already detailed in Sprint 2, builds a prompt, calls the language model, and parses and sanitises the returned customisation object. The **suggestion** workflow (Figure 4.19) takes a viewed product, reasons over the catalogue through a

read-only SQL tool, and returns, through a structured-output parser that enforces a `{betterDeal, alternatives}` shape, the identifiers of a cheaper same item and a few substitute alternatives, which the backend validates before serving (detailed below).

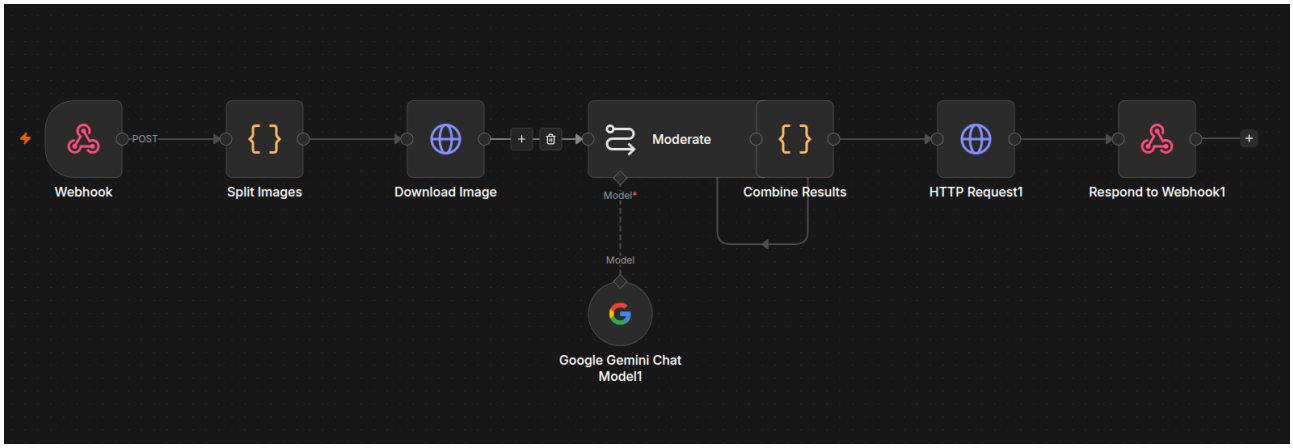

Figure 4.16: n8n workflow, AI verification.

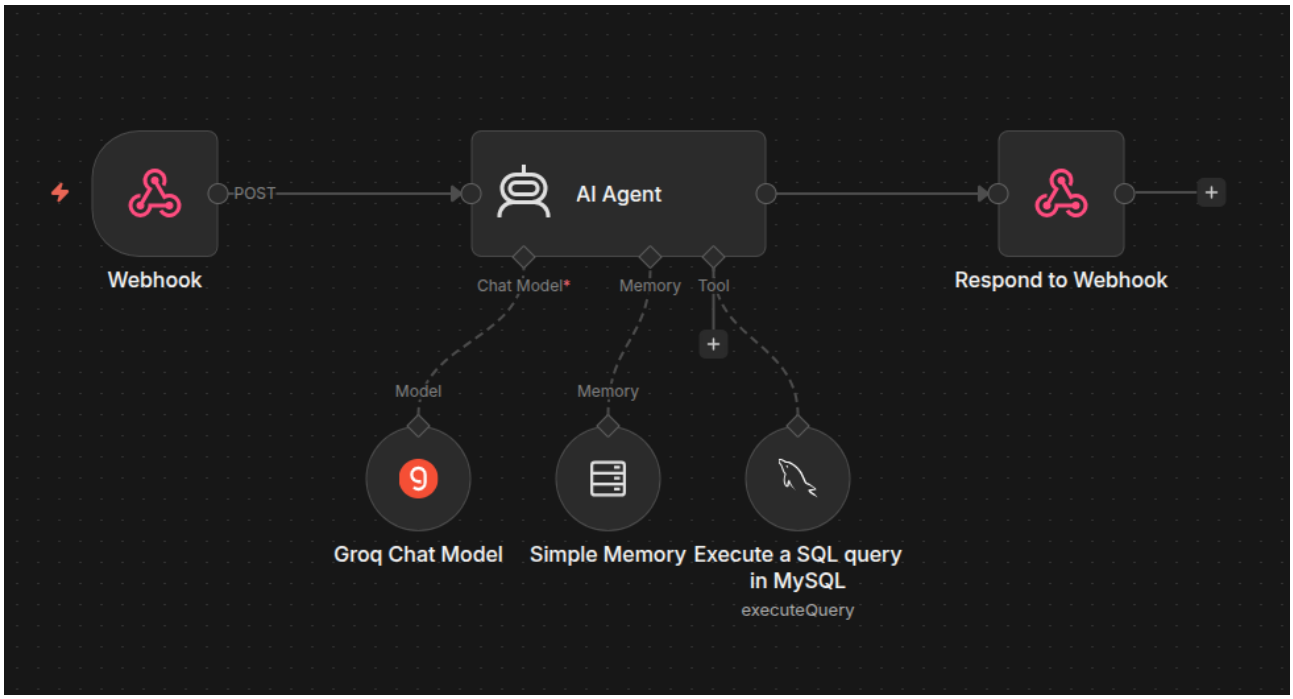

Figure 4.17: n8n workflow, database chat assistant.

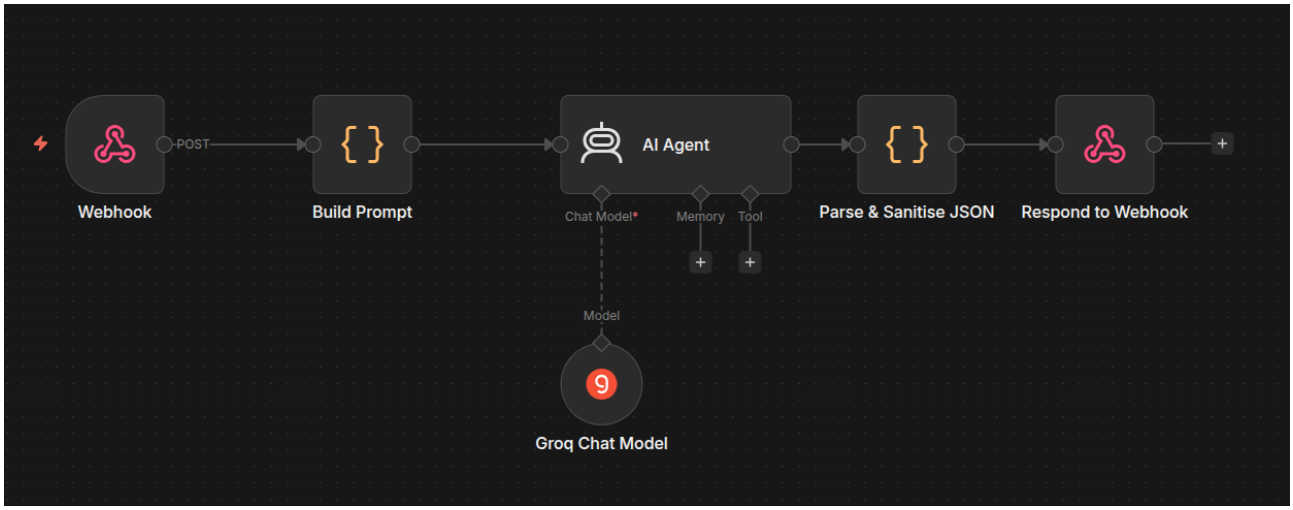

Figure 4.18: n8n workflow, AI storefront redesign.

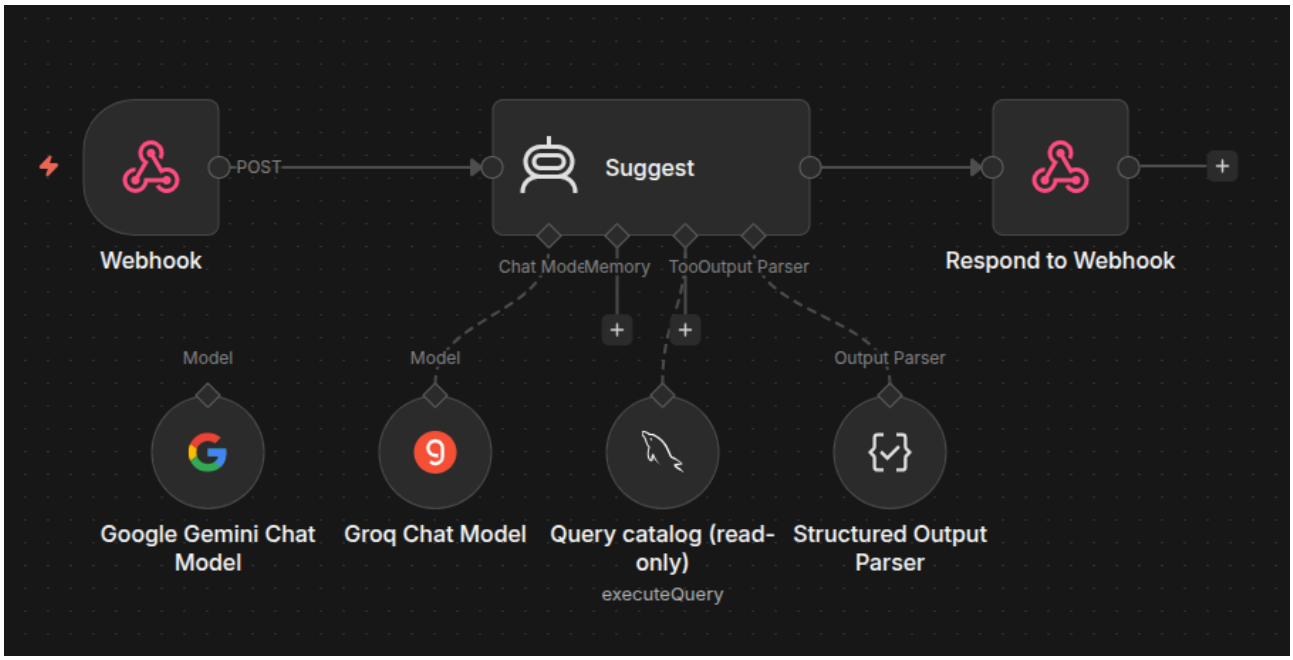

Figure 4.19: n8n workflow, product suggestions.

4.4.5 Product Suggestions

On every product page the platform surfaces two tiers of cross-store suggestion, a Tier 1 "same item, cheaper" offer and a Tier 2 set of substitute *alternatives*, distinct from the tag-based landing-page recommendations. They are produced **live, on each product view** rather than precomputed: an initial design that stored a vector embedding per product and compared them in-app was deliberately dropped in favour of the live workflow, so a newly listed product is eligible immediately and no stored data can go stale. The frontend requests them on a separate asynchronous query, so the product page paints at once and only the suggestion section waits.

The endpoint `GET /product/:id/suggestions` calls the suggestion workflow (Figure 4.19) with the viewed product, bounded by a timeout, and receives a structured `{betterDeal, alternatives}`

list of product ids. Because that output originates from a language model, it crosses a trust boundary and is **validated deterministically at serve time**: every id is hydrated and dropped unless it is approved, not banned, available, in stock and different from the viewed product, and the better deal is kept only when its price is genuinely below the viewed one.

On any workflow error, timeout, or empty or malformed response the backend falls back to a **deterministic catalogue query**: eligible products in the same category, ranked by a similarity score that weighs shared product-name tokens most, then shared tags, then the same sub-category, with popularity breaking ties. The fallback guarantees the section is never empty on a transient failure, while the live workflow, when available, adds semantic reasoning over the product name and description that a rigid query cannot. Figure 4.20 traces the request from the product page through the workflow call, serve-time validation, and the fallback.

[Screenshot to insert: Sequence diagram, the product page requests suggestions; the backend calls the workflow, validates and hydrates the returned ids, and falls back to the deterministic catalogue query on any failure]

Figure 4.20: Sequence diagram, product-suggestions request, serve-time validation and fallback.

4.4.6 Premium Tier

Unlike the one-time checkout of Sprint 3, the premium tier is a *recurring monthly subscription*, and it therefore introduces two Stripe mechanisms beyond that sprint's single PaymentIntent: **Stripe Checkout in subscription mode** for sign-up and the **Stripe Customer Portal** for self-service management. A seller starts the flow from the upgrade page; the backend creates, or reuses , a Stripe *customer* for the account, stamps its identifier on `stripeCustomerId`, and opens a hosted Checkout session in subscription mode for the configured monthly price. Everything afterwards , cancelling, updating the card, downloading invoices, is delegated to the hosted Customer Portal rather than rebuilt in-house.

The tier is **sellers-only**: the `premium/checkout` endpoint is role-gated to the seller role, a non-seller is routed instead to the become-a-seller flow, and administrators keep a **manual grant** comp path that marks an account premium without any Stripe subscription.

Entitlement is **webhook-driven**. The subscription lifecycle events , `customer.subscription.created`, `...updated` and `...deleted`, arrive through the *same* signature-verified `/payments/webhook` introduced in Sprint 3. A created or updated subscription sets `isPremium` and stamps `premiumExpiresAt` with the current period end, so each successful renewal simply advances the expiry; a scheduled cancellation is recorded in `cancelAtPeriodEnd` without revoking access, so premium is kept until the paid period actually ends; a deleted subscription revokes premium and detaches the subscription id. To carry this, the `User` entity gains `stripeCustomerId`, `stripeSubscriptionId` and `cancelAtPeriodEnd` alongside the existing `isPremium` and `premiumExpiresAt` (Figure 4.2); the premium-status read returned to the client now also exposes `cancelAtPeriodEnd` and a `manageable` flag that tells a real subscriber, who can open the portal, apart from an administrator comp grant.

Premium unlocks the full storefront customisation set, free-form colours, all fonts and radii, banner controls and the rest, and the ability to operate several stores. As before, the design deliberately *preserves* a seller's premium customisation values after expiry: the stored design remains visible to buyers and only the editing of premium fields is locked, so a lapsed subscription never degrades a storefront that is already live (US27). Figure 4.21 traces the subscription exchange from checkout through the webhook-driven entitlement update.

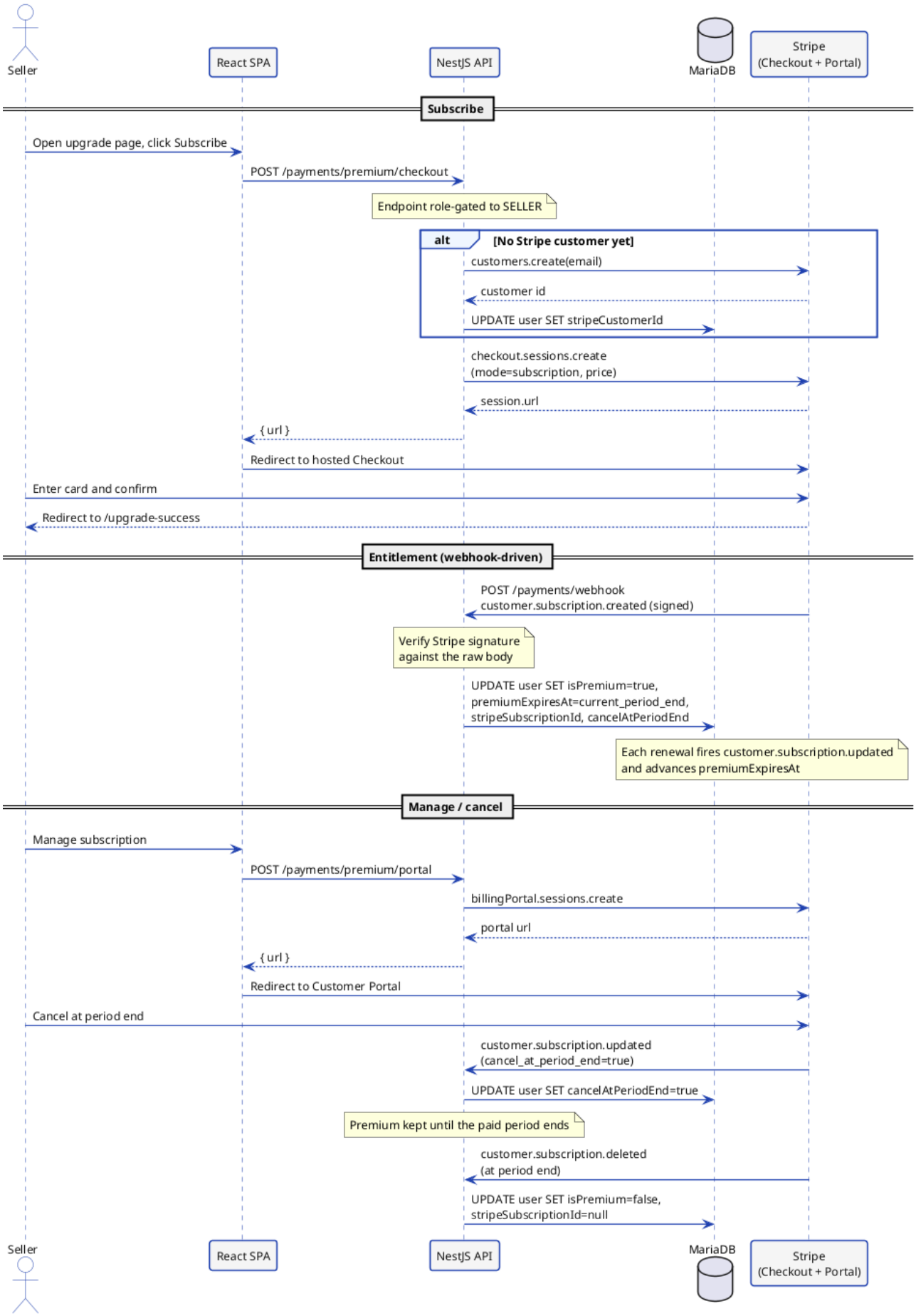

Figure 4.21: Sequence diagram, premium subscription checkout and webhook-driven entitlement.

4.4.7 Realisation

The sprint delivers the interfaces shown below. Each frame is a placeholder for the corresponding screenshot of the running application.

[Screenshot to insert: AI verification panel, entity verdict, rationale and admin override controls]

Figure 4.22: AI verification and moderation panel.

[Screenshot to insert: AI chat assistant, natural-language database query and formatted answer]

Figure 4.23: AI database-chat assistant.

[Screenshot to insert: Product page, the "Same item, cheaper" offer and the "Alternatives" row across stores, with the brief loading state shown while the live suggestions are fetched]

Figure 4.24: Cross-store product suggestions on the product page.

[Screenshot to insert: Upgrade page, the premium plan with the subscribe call to action, and, for an active subscriber, the manage-subscription controls (cancel at period end, update card, invoices)]

Figure 4.25: Premium upgrade and subscription-management page.

[Screenshot to insert: Appearance, the same page in the light theme and in the dimmed dark theme, with the theme switch open in the navigation bar]

Figure 4.26: Light and dimmed-dark application themes.

Across both themes the application carries a deliberate visual identity, applied entirely through CSS design tokens: a **warm-coral** brand accent in the light theme, a dimmed dark theme for low-light use, and **Inter** as the interface typeface. Centralising colour, typography and radius as tokens lets every component re-theme instantly and keeps the brand consistent across the application chrome, while seller storefronts intentionally retain their own chosen palette.

4.5 Tests & Validation

Quality is enforced by an automated suite of **538 tests** that runs on every push and gates every deployment.

4.5.1 Testing Strategy and Tooling

The backend is tested with **Jest 30** [21] and `ts-jest`, which transforms TypeScript on the fly and runs every co-located `*.spec.ts` file; collaborators such as the Prisma service, the notification service, the

SMTP mailer and the authentication guards are replaced with deterministic mocks. The frontend is tested with **Vitest** [22] and React Testing Library with `jest-dom`, which mount components, query the DOM by accessible role and text, and assert on the rendered output. Good tests here verify observable behaviour, the result of a service call, the response of an endpoint, the rendered state of a component, rather than internal implementation details, so they survive refactoring.

4.5.2 Test Pyramid

The suite follows the classical pyramid: a wide base of fast pure-logic and frontend tests, a thicker middle of service-level tests that run the real service code against a mocked Prisma client, and a narrow top of integration tests that exercise the wired-up HTTP stack through supertest and the global deny-by-default guard. Figure 4.27 shows how the tests distribute across these layers.

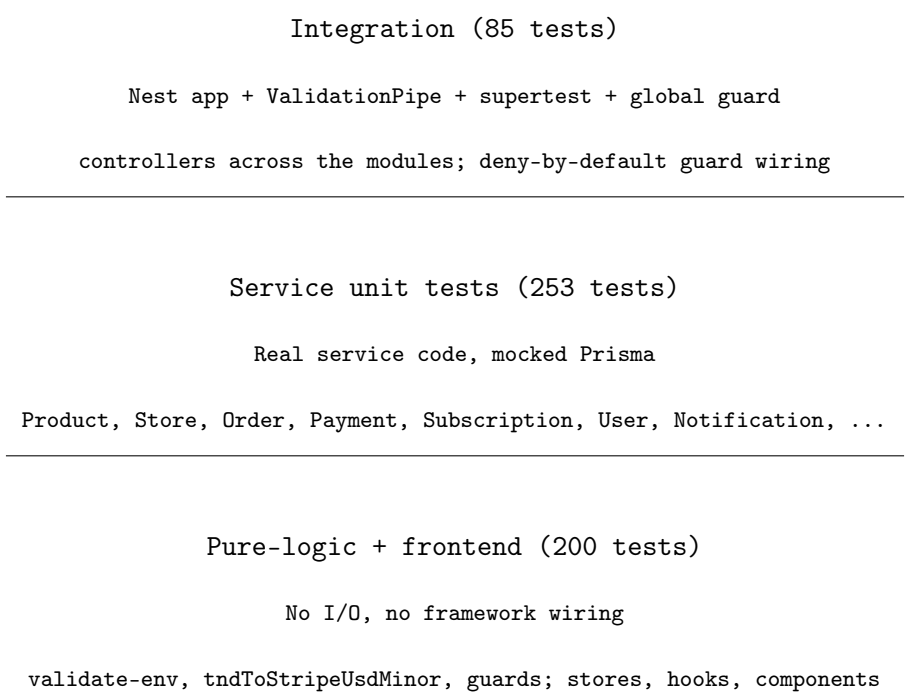

Figure 4.27: Test pyramid, 538 tests across backend (Jest, 391) and frontend (Vitest, 147).

4.5.3 Backend Unit and Integration Tests

Backend unit tests inject an in-memory mock of the Prisma service into the real service under test through the NestJS `TestingModule`, which lets each test drive arbitrary return values and assert on the exact arguments passed to Prisma, validating business rules such as ownership enforcement, the server-side payment-amount derivation and the stale-checkout reconciliation. The product-suggestions logic is covered the same way: the serve-time validation and hydration, the deterministic fallback ranking and the structured-output parsing are unit-tested against mocked workflow responses, and the endpoint is exercised with the suggestion webhook mocked so the happy path, the eligibility drops and

the fallback are all asserted without any network call. Integration tests boot the Nest application with its validation pipe and global guard and drive real controllers through supertest, confirming that the deny-by-default authorisation wiring admits exactly the intended routes.

4.5.4 Frontend Tests

The frontend tests cover the Zustand stores, the custom hooks and the React components, including the theme switch and the centralised status-badge helper introduced with the light/dark theming, mounting them in a jsdom environment and asserting on accessible, user-visible output. Together with the backend suite they give the layered coverage summarised in the pyramid above.

4.5.5 Code Coverage and Continuous Integration

Both runners collect coverage, and the whole suite is wired into the continuous-integration pipeline: every push runs linting, a build and the full test run for the backend and the frontend, and a red suite blocks the build-and-deploy jobs that follow (Section 4.6.3). Figure 4.28 stands in for the CI run and coverage summary.

[Screenshot to insert: Continuous-integration run, test and coverage summary for backend and frontend]

Figure 4.28: Continuous-integration test and coverage summary.

4.6 DevOps & Deployment

The platform is delivered as a set of containers, provisioned on reproducible infrastructure and shipped by an automated pipeline. The logical and physical architecture were established in Section 1.4; this section details the realisation of the delivery chain.

4.6.1 Containerisation

Each service ships as a Docker image. The backend uses a **multi-stage** build (Listing 4.2): a builder stage installs dependencies, generates the Prisma client and compiles the application, then prunes development dependencies, and a slim production stage copies only the build artefacts and runs as the

unprivileged `node` user, applying database migrations on start.

```
# build stage
FROM node:20-alpine AS builder
RUN corepack enable && corepack prepare pnpm@10 --activate
WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN pnpm install --frozen-lockfile
COPY . .
RUN npx prisma generate && pnpm run build && pnpm prune --prod

# production stage
FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/prisma ./prisma
USER node
EXPOSE 3000
CMD ["sh", "-c", "npx prisma migrate deploy && node dist/src/main"]
```

Listing 4.2: Backend multi-stage Dockerfile (excerpt).

The services are composed with Docker Compose: a MariaDB database with a health check, the backend, the frontend, an n8n engine (with a one-shot initialiser that imports the version-controlled workflows), and a **Caddy** [23] reverse proxy that terminates TLS with automatically renewed Let's Encrypt certificates and adds the HTTP security headers. Listing 4.3 shows the backend and proxy service blocks.

```
services:
  backend:
    image: registry/marketplace-backend:${IMAGE_TAG:-latest}
    restart: always
    depends_on:
      db:
        condition: service_healthy
    environment:
      DB_HOST: db
      NODE_ENV: production
      STRIPE_SECRET_KEY: ${STRIPE_SECRET_KEY}
```

```
      STRIPE_WEBHOOK_SECRET: ${STRIPE_WEBHOOK_SECRET}
      N8N_VERIFY_WEBHOOK_URL: http://n8n:5678/webhook/ai-verify
    volumes:
      - uploads:/app/uploads

  caddy:
    image: caddy:2-alpine
    restart: always
    ports: ["80:80", "443:443"]
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
```

Listing 4.3: Docker Compose backend and reverse-proxy services (excerpt).

4.6.2 Infrastructure as Code

The production environment is described declaratively with **Terraform** using the DigitalOcean [24] provider, so the whole server can be recreated identically with a single `terraform apply` and every change is reviewable as a diff. Three resources define the environment: a private `digitalocean_vpc`, an `ubuntu-24-04-x64` `digitalocean_droplet` attached to it with the operator's SSH key, and a firewall configured through the droplet's `user_data` cloud-init script. That script makes the first boot zero-touch: it installs Docker Engine and the Compose plugin, then configures UFW to *deny all inbound by default* while allowing only SSH, HTTP, HTTPS and the n8n webhook port. The provider also manages the DNS records that point the domain at the droplet.

4.6.3 CI/CD Pipeline

A **GitHub Actions** [25] workflow turns a push to `main` into a live deployment (Listing 4.4). It first runs the backend and frontend test jobs; only if both pass does it provision the infrastructure with Terraform, build and push the backend and frontend Docker images to the registry, and finally deploy by copying the Compose file to the droplet over SSH and bringing the stack up. Secrets are injected from the CI secret store, so no credential lives in the repository, and DNS is updated to the droplet's address automatically.

```
name: Build, Push & Deploy
on:
  push:
    branches: [main]
```

```
jobs:
  test-backend:    # pnpm install, prisma generate, lint, build, jest
    runs-on: ubuntu-latest
  test-frontend:   # pnpm install, lint, build, vitest
    runs-on: ubuntu-latest
  provision:       # terraform init + apply (droplet, firewall, DNS)
    runs-on: ubuntu-latest
  build-backend:   # docker build + push to the registry
    needs: [test-backend, test-frontend]
    runs-on: ubuntu-latest
  deploy:
    needs: [provision, build-backend, build-frontend]
    runs-on: ubuntu-latest
    steps:
      - name: Deploy over SSH
        run: ssh $HOST "docker compose pull && docker compose up -d"
```

Listing 4.4: GitHub Actions build-and-deploy pipeline (excerpt).

Conclusion

Release 2 completed the product. Sprint 3 made the marketplace transactable, a multi-store cart and checkout, a server-authoritative Stripe flow confirmed by a signed webhook, self-service account management, and administration and moderation tooling. Sprint 4 added the intelligence and delivery layers, automated AI verification with administrative override, the database chat assistant, the premium tier, light and dimmed-dark theming, a 538-test suite, and a containerised, Terraform-provisioned deployment driven by a GitHub Actions pipeline. The marketplace planned in the first two chapters is now fully realised, validated and deployable.

General Conclusion

This report has presented the design and development of a multi-vendor e-commerce marketplace, carried out as a final-year engineering project within the host company. The work set out to address concrete gaps in the local e-commerce landscape, heavyweight seller onboarding, the absence of integrated payment, and the lack of any quality moderation, with a platform on which independent sellers open a store, publish a catalogue, and sell to customers under a single, trustworthy roof.

The first two chapters framed the problem and the solution. After situating the project in its organisational and competitive context, we analysed the requirements, identified the actors, and specified the functional and non-functional needs, which we modelled with UML use-case and class diagrams. The Scrum framework then organised the work into a product backlog of forty-one user stories and a plan of two releases of two sprints each, underpinned by the architecture and technical choices that frame the realisation.

The last two chapters delivered that plan. **Release 1** built the foundations of the marketplace: an identity layer with email verification, role-based access and database-backed sessions; the seller-onboarding loop of application and administrative review; and the core selling capabilities, store creation, a seven-section WYSIWYG storefront editor with AI-assisted redesign, product management, and store-wide sales. **Release 2** turned the populated catalogue into a complete product: a multi-store cart and checkout with a server-authoritative Stripe payment flow confirmed by a signature-verified webhook; the administration and moderation tooling; an automated AI verification pipeline with administrative override and a natural-language database-chat assistant; a premium subscription tier; and the quality and delivery layers, a layered suite of 538 automated tests, containerised services behind a TLS-terminating reverse proxy, infrastructure described as code with Terraform, and a continuous deployment pipeline that tests, builds and ships on every push to the main branch.

Beyond the deliverable itself, the project was an opportunity to practise the full software-engineering cycle end to end, from requirements analysis and agile planning to test-driven implementation, AI integration and modern DevOps, and to make pragmatic engineering decisions under real constraints, such as deriving payment amounts on the server, screening user content automatically, and keeping the whole environment reproducible.

Several directions remain open for future work. The features deliberately left out of scope, a native mobile application, multi-currency settlement, advanced fraud detection, an on-premise language model, and a dedicated full-text search engine, are natural next steps. The AI verification and redesign workflows could be extended with richer models and human-in-the-loop feedback, the analytics could

grow into seller- and marketplace-level dashboards, and the single-droplet deployment could evolve towards a horizontally scaled, observable production environment. Together these would carry the platform from a complete, validated prototype towards a production marketplace at scale.

Bibliography

[1] K. Beck et al. “Manifesto for agile software development.” [Accessed 2026]. [Online]. Available: https://agilemanifesto.org/

[2] K. Schwaber and J. Sutherland. “The scrum guide: the definitive guide to scrum.” [Accessed 2026]. [Online]. Available: https://scrumguides.org/

[3] Object Management Group. “Unified modeling language (uml) specification, version 2.5.1.” [Accessed 2026]. [Online]. Available: https://www.omg.org/spec/UML/

[4] Google. “Gemini api — google ai for developers.” [Accessed 2026]. [Online]. Available: https://ai.google.dev/

[5] Groq. “Groq: fast ai inference.” [Accessed 2026]. [Online]. Available: https://groq.com/

[6] GitHub. “Github: build and ship software on a single, collaborative platform.” [Accessed 2026]. [Online]. Available: https://github.com/

[7] N. Labs. “Notion: the connected workspace.” [Accessed 2026]. [Online]. Available: https://www.notion.so/

[8] Microsoft. “Visual studio code: code editing. redefined.” [Accessed 2026]. [Online]. Available: https://code.visualstudio.com/

[9] I. Docker. “Docker: accelerated container application development.” [Accessed 2026]. [Online]. Available: https://www.docker.com/

[10] HashiCorp. “Terraform: infrastructure as code.” [Accessed 2026]. [Online]. Available: https://www.terraform.io/

[11] n8n GmbH. “N8n: secure workflow automation for technical teams.” [Accessed 2026]. [Online]. Available: https://n8n.io/

[12] Postman. “Postman: the collaboration platform for api development.” [Accessed 2026]. [Online]. Available: https://www.postman.com/

[13] pnpm. “Pnpm: fast, disk space efficient package manager.” [Accessed 2026]. [Online]. Available: https://pnpm.io/

[14] Microsoft. “Typescript: javascript with syntax for types.” [Accessed 2026]. [Online]. Available: https://www.typescriptlang.org/

[15] M. O. Source. “React: the library for web and native user interfaces.” [Accessed 2026]. [Online]. Available: https://react.dev/

[16] K. Mysliwiec. “Nestjs: a progressive node.js framework.” [Accessed 2026]. [Online]. Available: https://nestjs.com/

[17] Prisma. “Prisma orm: next-generation node.js and typescript orm.” [Accessed 2026]. [Online]. Available: https://www.prisma.io/

[18] Better-Auth. “Better-auth: authentication framework for typescript.” [Accessed 2026]. [Online]. Available: https://www.better-auth.com/

[19] Craft.js. “Craft.js: a react framework for building drag-and-drop page editors.” [Accessed 2026]. [Online]. Available: https://craft.js.org/

[20] Stripe. “Stripe payments and the payment intents api.” [Accessed 2026]. [Online]. Available: https://stripe.com/docs/payments

[21] M. O. Source. “Jest: delightful javascript testing.” [Accessed 2026]. [Online]. Available: https://jestjs.io/

[22] V. Team. “Vitest: next generation testing framework.” [Accessed 2026]. [Online]. Available: https://vitest.dev/

[23] Caddy. “Caddy: the ultimate server with automatic https.” [Accessed 2026]. [Online]. Available: https://caddyserver.com/

[24] DigitalOcean. “Digitalocean: cloud infrastructure for developers.” [Accessed 2026]. [Online]. Available: https://www.digitalocean.com/

[25] GitHub. “Github actions: automate your workflow from idea to production.” [Accessed 2026]. [Online]. Available: https://github.com/features/actions

Résumé

Ce rapport présente la conception et le développement d'une place de marché électronique multi-vendeurs, réalisé dans le cadre d'un projet de fin d'études au sein de la société NeuralBey. La plateforme permet à des vendeurs indépendants d'ouvrir leurs propres boutiques, de publier des produits et de traiter des commandes au sein d'un environnement partagé et gouverné de façon centralisée, tandis que les administrateurs valident les vendeurs, modèrent le contenu et mettent en avant une sélection de qualité. Sa principale particularité est un pipeline de vérification assisté par l'intelligence artificielle qui contrôle automatiquement les boutiques et les produits afin de préserver la confiance des acheteurs à grande échelle, complété par un assistant conversationnel de base de données et une refonte de vitrine assistée par IA. Le système est bâti comme une application React monopage adossée à une API modulaire NestJS et une base de données MariaDB, n8n orchestrant les traitements d'intelligence artificielle, et il est livré au moyen d'une chaîne DevOps entièrement conteneurisée et décrite comme code (Docker, Terraform, GitHub Actions). Le projet a suivi une démarche agile Scrum répartie sur deux releases.

Mots clés : Place de marché multi-vendeurs, E-commerce, Intelligence Artificielle, NestJS, React, Scrum, DevOps

Abstract

This report presents the design and development of a multi-vendor e-commerce marketplace, carried out as a final-year engineering project at NeuralBey. The platform lets independent sellers open their own stores, publish products and fulfil orders within a shared, centrally-governed environment, while administrators onboard sellers, moderate content and curate featured selections. Its distinguishing feature is an AI-assisted verification pipeline that automatically screens stores and products to protect buyer trust at scale, complemented by a conversational database assistant and AI-assisted storefront redesign. The system is built as a React single-page application backed by a modular NestJS API and a MariaDB database, with n8n orchestrating the AI workflows, and is delivered through a fully containerised, infrastructure-as-code DevOps pipeline (Docker, Terraform, GitHub Actions). The project followed an agile Scrum approach spread over two releases.

Keywords : Multi-vendor marketplace, E-commerce, Artificial Intelligence, NestJS, React, Scrum, DevOps